

Introduction to Python libraries

There are **two Python Libraries in Your Syllabus** that you have to learn.

1. Pandas Library
 - a. Introduction to Data Structures in Pandas (DataFrame and Series)
2. Matplotlib Library
3. Most Important Questions and Answers

1. Pandas Library

- I. Python Pandas is an **Open Source Python Library**.
- II. Pandas has derived from “**Panel Data System**”.
- III. Pandas is a popular choice for **Data Analysis** and **Data Science**, because it is very simple and easy to use.
- IV. The main author of Pandas is **Wes McKinney**.
- V. Pandas offers **high-performance, easy-to-use** data structure and data analysis tools.

Note:-Data Analysis – Process of evaluating big datasets using analytical and statistical tools so as *to discover useful information and conclusions* to support business decision-making.

Data Science - Data Science is a blend of various tools, algorithms, and machine learning principles with the *goal to discover hidden patterns from the raw data*.

Why Pandas

Pandas is the most popular **library in the Scientific Python ecosystem** for doing data analysis. Pandas is capable of many tasks including:

- I. It can **read or write in many different data formats** (Integer, float, double etc).
- II. It can calculate in all ways data is organized, i.e. **row and column wise analysis**.
- III. It can easily select **subsets of data from bulky data sets** and even combine multiple datasets together.
- IV. It **supports visualization by integrating matplotlib** and seaborn etc libraries.

Pandas Data Structures

A **data structure** is a particular way of organizing data in a computer so that it can be used effectively. For example, we can store a list of items using the list data structure.

- I. There are **four built-in data structures in Python** - list, tuple, dictionary and set.
- II. Pandas offers many data structures to handle a variety of data. Out of many Data Structures pandas **two basic Data Structures of Pandas**

- A. **Series** - A Series is a Pandas data structure that represents a **one-dimensional array-like object** containing an array of **data** and an associated array of data labels, called its **index**.

Rahul	65
Purnojit	70
Suraj	66
Nikhil	50
dtype: int64	

- B. **DataFrame** - A DataFrame is a **two dimensional** labeled array like Pandas data structure that **stores an ordered collection columns** that can store data of different types.

	AGE	CLASS
Arijit	16	X
Baishali	17	XI
Adrija	18	XII

Note:- Both Series and DataFrame are objects in python.

Pandas Installation

- I. Pandas Library can be install in your python by opening command prompt and running the following command:

```
pip install pandas
```

- II. If the above command doesn't work try following commands, or [set environment variables for Python](#).

```
py -m pip install pandas python -m pip install pandas
```

- III. To use Pandas in your python program you have to import pandas:

```
import pandas
```

Note:- Make sure you have already installed Python on your Computer. To know how to download and install Python [click here](#).

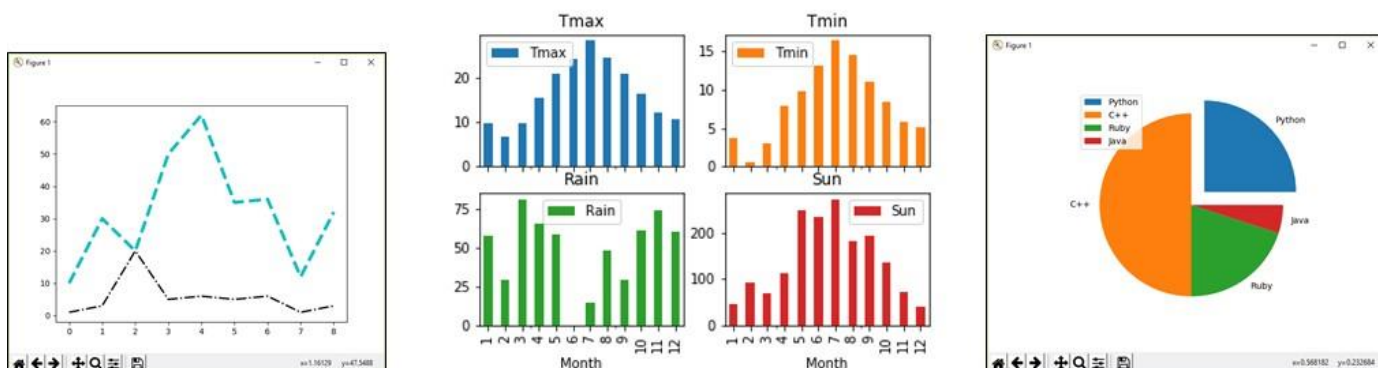
Internet connection is required to install python pandas library.

2. Matplotlib Library

There are multiple tools available for performing visualization in data science. In Python we can use **exclusive libraries for visualization**, commonly known as **matplotlib**.

Note:-

- **Data Visualization:-** It is the graphical representation of information and data. By using visual elements like charts, graphs, and maps, data visualization tools provide an accessible way to see and understand trends, outliers, and patterns in data.



Why Matplotlib

- I. Python based plotting library offers matplotlib with a **complete 2D support along with limited 3D graphic support**.
- II. It is useful in **producing publication quality figures** in interactive environments across platforms. It can also be **used for animations as well**.
- III. The **pyplot interface** of matplotlib is used for representing **2D graphs**, which we are going to cover in our syllabus.

Matplotlib Installation

- I. Pandas Library can be install in your python by opening command prompt and running the following command:

```
pip install matplotlib
```

- II. If the above command doesn't work try following commands, or [set environment variable for Python](#).

```
py -m pip install matplotlib python -m pip install matplotlib
```

Note:- Make sure you have already installed Python on your Computer. To know how to download and install Python [click here](#).

Internet connection is required to install python pandas library.

What is pyplot ?

pyplot is a **collection of command style functions** that is used for Data Visualization. It is available **inside matplotlib**.

1. pyplot interface provides many methods for 2D plotting of Data. Ex:- ***plot()***, ***bar()***, ***pie()***, ***hist()*** etc.
2. To use pyplot we have to import it by using:

```
import matplotlib.pyplot as plt
```

3. Multiple types of graphs can be plotted using pyplot ex:- ***line chart, bar chart, pie chart, scattered chart, histogram, box plot*** etc.
4. According to XII-IP revised syllabus 2022-23 you have to learn ***line plot, bar graph*** and ***histogram***.

Series

1. Introduction
2. Creation of Series Object
 - a. With different types of Data
 - b. Assigning User defined index to Series Object
 - c. Assigning data type to series object (dtype)
3. Series Attributes
4. Accessing and Changing Series Elements
5. Accessing and Changing Series Slicing
6. Mathematical Operations on Series
7. Basic Series Methods (head(), tail(), min(), max(), sum(), count(), mean())

1. Introduction to Series

- I. Series type Object has two main components:
 - A. An array of **actual data**
 - B. An associated array of **indexes of data labels**.
- II. Like lists there are indexes in Series, but in **series we can assign our own indexes** (Data labels). The **labels need not be unique** but must be a hashable type.

Index	Data
1	'A'
2	'B'
3	'C'
4	'D'
5	'E'

- III. To create the series we have to use Pandas function **Series()**.

2. Series Object Creation

Before creating Series Object we have to **import pandas library**

```
import pandas
```

Syntax for Series Creation:

```
<Series Object> = pandas.Series(data=None, index=None, dtype=None, name=None, copy=False, fastpath=False)
```

Data Parameter

According to your syllabus you must know the creation of the Series using **ndarray**, **dictionary**, **scalar value**. Now, let us discuss the creation of series using following ways:

1. Creation of **Empty Series**
2. Using Python Sequence (**List, Tuple**)
3. Using **ndarray**
4. Using Python **Dictionary**
5. Using a **Scalar Value**

1. Creation of Empty Series

```
import pandas as pd
s = pd.Series()
print(s)

Series([], dtype: float64)

print(type(s))

<class 'pandas.core.series.Series'>
```

2. Using Python Sequence (List, Tuple)

List:

```
import pandas as pd
myList = ['A','B',2]
s = pd.Series(data = myList)
print(s)

0    A 1    B 2    2
dtype: object
```

Tuple:

```
import pandas as pd
myTuple = ('A','B',2)
s = pd.Series(myTuple)
print(s)

0    A 1    B 2    2
dtype: object
```

3. Using ndarray

```
import numpy as np
import pandas as pd
arr = np.array([1,2,3,4])
s = pd.Series(arr)
print(s)

0    1
1    2
2    3
3    4
dtype: int32
```

4. Using Python Dictionary

```
import pandas as pd

d = {'A':1, 'B':2, 'C':3} s =
pd.Series(d)

print(s)
A    1
B    2
C    3
dtype: int64
```

5. Using a Scalar Value

```
import pandas as pd

s = pd.Series(20) print(s)

0    20
dtype: int64
```

index Parameter

- I. The Series method gives the **default indexes** as **0,1,2etc.** Now we are going to discuss how to provide user-defined indexes using the **index Parameter** of the Series Method.
- II. It is used to provide the **data labels** for data values.
- III. The **number of indexes should be the same as the number of data values.**

```
import pandas as pd l = [1,2,3] s = pd.Series(data= l, index =
['X','Y','Z']) print(s)
```

Output

```
X    1
Y    2
Z    3
dtype: int64
```

dtype parameter

dtype parameter is used to **provide the data type** to the Series elements. Since a **single series can have a single data type**, we can assign our own data type using dtype parameters.

```
import pandas as pd
import numpy as np
s = pd.Series(data = [1,2,3,4], dtype = np.int16)
print(s)
```

Output

```
0    1
1    2
2    3
3    4
dtype: int16
s = pd.Series(data = [1,2,3,4], dtype = np.float32)
print(s)
```

Output

```
-----
0    1.0
1    2.0
2    3.0
3    4.0
dtype: float32
```

3. Series Attributes

Attributes	Description
Series.index	Range of the index (axis labels) of the Series.
Series.values	Return Series as ndarray or ndarray like depending upon dtype
Series.dtype	Return the dtype object of the underlying data.
Series.shape	Return a tuple of the shape of the underlying data.
Series.nbytes	Return the number of bytes in the underlying data.
Series.ndim	Number of dimensions of the underlying data, by definition 1.
Series.size	Return the number of elements in the underlying data.
Series.hasnans	Return if I have any nans; enables various perf speedups.
Series.empty	Return true if Series is empty
at, iat	To access a single value from Series
loc, iloc	To access slices from Series

Let us see each attribute with programming **example**:

```
import pandas as pd
s = pd.Series(data = [1,2,3,4], dtype = np.int16)
print(s)
```

```
0    1
1    2
2    3 3  4 dtype: int16
```

```
# ## Series.index
s.index
```

```
RangeIndex(start=0, stop=4, step=1)
```

```
# ## Series.values
s.values array([1, 2, 3, 4], dtype=int16)
```

```
# ## Series.dtype
s.dtype dtype('int16')
```

```
# ## Series.shape
s.shape (4,)
```

```
# ## Series.nbytes
s.nbytes 8
```

```
# ## Series.ndim
s.ndim
1
```

```
# ## Series.size
s.size 4
```

```
# ## Series.hasnans
s.hasnans False
```

```
# ## Series.empty
s.empty False
```

```
# ## Series.at / iat
s.at[0]
1
```

4. Accessing and Changing Series Elements

In the previous topics you have seen how to create a Series. Now we will discuss how you can access the elements of the series and how we can change them.

Consider the following Series “s”:

```
import pandas as pd
s = pd.Series(data = [1,2,3,4], index = ['A','B','C','D'])
print(s)
```

```
A    1
B    2
C    3
D    4
dtype: int64
```

Now, There are **two ways using which you can access the series elements:**

I. By using data labels (index)

```
s['A'] 1

# ## Changing Series Elements
s['A'] = 99
print(s)
A    99
B     2
C     3
D     4
dtype: int64
```

II. By using index position

```
s[2] 3

# ## Changing Series Elements
S[2] = 99
print(s)
A     1
B     2
C    99
D     4
dtype: int64
```

Note:- Series internally maintains positions starting from 0 till **length – 1** .

5. Accessing and changing Series Slices

Slicing means extracting the **part of the Series**. Slicing can be done through following ways:

- 1) Using **indexing** operator([**start : stop : step**])
 - a) Position wise (slicing includes **stop - 1** data)
 - b) Data label wise (slicing includes **both ends**)
 - i) With **unique data labels**
 - ii) With **duplicate data labels**
- 2) Using **.loc** attribute
- 3) Using **.iloc** attribute

Now, let us see each type one by one:

1) Using indexing operator([start : stop : step])

We can use index position (0,1,2,3) as well as actual indexes (data labels) to find the slices.

a) Position wise (slicing includes stop - 1 data)

```
import pandas as pd
s = pd.Series(data = [1,2,3,4])
print(s)
```

```
0    1
1    2
2    3
3    4
dtype: int64
```

```
s[0:3]
```

```
0    1
1    2
2    3
dtype: int64
```

```
s[3:1:-1]
```

```
3    4
2    3 dtype: int64
```

```
# ## Changing Series Elements
```

```
s[0:3] = 99
```

```
0    99
1    99
2    99
3     4
dtype: int64
```

b) Data label wise (slicing includes both ends)

Since data labels can be duplicate there can be two cases.

i) With unique data labels

```
import pandas as pd
s = pd.Series(data = [1,2,3,4], index = ['A','B','C','D'])
print(s)
```

```
A    1
B    2
C    3
D    4
dtype: int64
```

```
# ## Changing Series Elements s['A':'B'] =
```

```
99
A    99
B    99
3D    4 dtype: int64
```

```
s['A':'D']
```

```
A    1
B    2
C    3
D    4
dtype: int64
```

```
s['A':'Z']
```

```
B     1
C     2
3D    4 dtype: int64
```

ii) With duplicate data labels

```

import pandas as pd s = pd.Series(data = [1,2,3,4,5], index = ['A','B','C','D','A'])
print(s)
A    1
B    2
C    3
D    4
A    5
dtype: int64
# non-unique values breaks slicing
s['A':'D']
KeyError: "Cannot get left slice bound for non-unique label: 'A'"
# while unique values are still slicable in a non-unique index
s['B':'D']
B    2
C    3
D    4
dtype: int64
s['B':'D':2]
B    2
D    4
dtype: int64
# ## Changing Series Elements
s['B':'D':2] = 99
A    1
B    99
C    3
D    99
A    5
dtype: int64

```

2) Using .loc attribute

loc attribute of Series is used to access a group of rows and columns by label(s) or a Boolean array.

Syntax:

i) <Series>.loc[start : stop : step] ii) <Series>.loc[[<list of labels>]]

Examples:

```

import pandas as pd import numpy as np s = pd.Series(data = [1,2,3,4,5], index = ['A','B','C','D','A'])
print(s)
A    1
B    2
C    3
D    4
A    5
dtype: int64

```

```

s.loc['D':2]
A    1
C    3
dtype: int64
s.loc[['A','B']]
A    1
A    5
B    2
dtype: int64
# ## Changing Series Elements
s.loc[['A','B']] = 99
A    99
B    99
C     3
D     4
A    99
dtype: int64

```

3) Using .iloc attribute

Purely integer-location based indexing for selection by position.

Syntax:

```
i) <Series>.iloc[ start : stop : step ] ii) <Series>.iloc[[<list of positions>]]
```

Examples:

```

import pandas as pd import numpy as np s = pd.Series(data = [1,2,3,4,5], index = ['A','B','C','D','A'])
print(s)
A    1
B    2
C    3
D    4
A    5
dtype: int64
s.iloc[0:4:2]
A    1
C    3
dtype: int64
s.iloc[[0,0,1,1]]
A    1
A    1
B    2
B    2
dtype: int64
# ## Changing Series Elements s.iloc[0:4] = 'X' print(s)
A    X
B    X
C    X
D    X
A    5
dtype: object

```

6. Mathematical Operations on Series

- I. You can perform arithmetic operations like **addition, subtraction, multiplication** etc. on two Series objects.
- II. The **operations are performed only on the matching indexes**.
- III. For all **non matching indexes** NaN (Not a Number) will be returned.

Note:- Like numpy array, **series support vector operations**.

A batch operations on data without writing any for loops. This is usually called **vectorization**.

```
# ## vectorization import pandas as pd
s = pd.Series(data = [1,2,3,4])
print(s)
```

```
0    1
1    2
2    3 3    4 dtype: int16
```

```
s*2
0    2
1    4
2    6 3    8 dtype: int16
```

Let us consider following two Series **S1** and **S2**. We will perform mathematical operations on these Series.

```
import pandas as pd
```

```
s1 = pd.Series(data = [1,2,3,4,5], index = ['A','B','C','D','E'])
s2 = pd.Series(data = [1,2,3,4,5], index = ['B','C','D','E','F'])
```

```
print(s1)
A    1
B    2
C    3
D    4
E    5
dtype: int64
print(s2)
B    1
C    2
D    3
E    4
F    5
dtype: int64
```

Now let us perform mathematical operations on these two Series.

```

s1 + s2
A  NaN
B  3.0
C  5.0
D  7.0
E  9.0
F  NaN
dtype: float64
s1 > 2
A  False
B  False
C  True
D  True
E  True
dtype: bool
# ## Filtering Entries
s1[s1 > 2]
C    3
D    4
E    5
dtype: int64

```

7. Basic Series methods (head(), tail(), min(), max(), sum(), count(), mean())

Consider the following Series s.

```

import pandas as pd
s = pd.Series(data = [1,2,3,4,5], index = ['A','B','C','D','E'])
print(s)
A    1
B    2
C    3
D    4
E    5
dtype: int64

```

Let us now discuss examples of different methods/functions on the series s.

head()

```

## gives the top <number> rows. By default 5.Syntax: head(<number>)
s.head()
A    1
B    2
C    3
D    4
E    5
dtype: int64
s.head(2)
A    1
B    2
dtype: int64
s.head(-2)
A    1
B    2

```

```
c 3
```

tail()

gives top <number> rows. By default 5.

Syntax: tail(<number>)

s.tail()

```
A 1
```

```
B 2
```

```
C 3
```

```
D 4
```

```
E 5
```

```
dtype: int64
```

s.tail(2)

```
D 4
```

```
E 5
```

```
dtype: int64
```

s.tail(-2)

```
C 3
```

```
D 4
```

```
E 5
```

```
dtype: int64
```

min()

```
s.min() 1
```

max()

```
s.max() 5
```

sum()

```
s.sum() 15
```

count()

```
s.count() 5
```

mean()

```
s.mean() 3.0
```

DataFrame

1. Introduction to DataFrame
2. DataFrame Object Creation
 - a. From Dictionary of List/Series
 - b. From List of List/Dictionaries
 - c. Text/CSV files
3. DataFrame attributes
4. Operations on rows and columns - select, add, delete and rename
5. Basic DataFrame methods (head(), tail(), min(), max(), sum(), count(), mean())
6. Importing/Exporting Data between CSV files and Data Frames.
7. Most Important Questions and Answers

1. Introduction to DataFrame

It is a **2 dimensional data structure** with columns of different types. It is just similar to a spreadsheet or SQL table, or a dict of Series objects.

The diagram shows a DataFrame table with 5 rows and 5 columns. The columns are labeled 'first_name', 'last_name', 'age', 'Comedy_Score', and 'Rating_Score'. The rows are indexed from 0 to 4. A bracket on the left labeled 'Rows' spans the row indices. A bracket at the top labeled 'Columns' spans the column headers. A bracket at the bottom labeled 'Data' spans the data cells. Several cells are circled: 'Raj' in row 1, column 1; '36' in row 2, column 2; '8' in row 3, column 3; and '5' in row 4, column 3.

	first_name	last_name	age	Comedy_Score	Rating_Score
0	Sheldon	Copper	42	9	25
1	Raj	Koothrappali	38	7	25
2	Leonard	Hofstadter	36	8	49
3	Howard	Wolowitz	41	8	62
4	Amy	Fowler	35	5	70

Characteristics of DataFrame Object:

- I. It has two indexes or axis **row index (axis = 0)** and **column index (axis = 1)**
- II. **Row index** is known as **index** and **column index** is known as **column name**
- III. Index(Row-Index) or Column (Column-Index) **can be numbers or letters or stings**
- IV. A column can have values of different types.
- V. DataFrame is **Value Mutable** and **Size Mutable**

2. DataFrame Object Creation

Before creating DataFrame Object we have to **import pandas library**

```
import pandas
```

Syntax for DataFrame Creation:

```
<df_object> = pandas.DataFrame(data = None, index = None, columns = None, dtype = None, copy = False)
```

1. From dictionary of List/Series
2. From List of List/Dictionaries
3. From Text/CSV files

data parameter

1. From Dictionary of List/Series

Dictionary of List:

```
import pandas as pd

d = {'first_name': ['Sheldon', 'Raj', 'Leonard', 'Howard', 'Amy'],
     'last_name': ['Copper', 'Koothrappali', 'Hofstadter', 'Wolowitz',
                  'Fowler'],
     'age': [42, 38, 36, 41, 35],
     'Comedy_Score': [9, 7, 8, 8, 5],
     'Rating_Score': [25, 25, 49, 62, 70]}

df = pd.DataFrame(d)
print(df)
```

Output

	first_name	last_name	age	Comedy_Score	Rating_Score
0	Sheldon	Copper	42	9	25
1	Raj	Koothrappali	38	7	25
2	Leonard	Hofstadter	36	8	49
3	Howard	Wolowitz	41	8	62
4	Amy	Fowler	35	5	70

Dictionary of Series:

```
import pandas as pd
d = {'first_name': pd.Series(['Sheldon', 'Raj', 'Leonard', 'Howard', 'Amy']),
     'last_name': pd.Series(['Copper', 'Koothrappali', 'Hofstadter', 'Wolowitz', 'Fowler']),
     'age': pd.Series([42, 38, 36, 41, 35]),
     'Comedy_Score': pd.Series([9, 7, 8, 8, 5]),
     'Rating_Score': pd.Series([25, 25, 49, 62, 70])}
df = pd.DataFrame(d)
print(df)
```

Output

	first_name	last_name	age	Comedy_Score	Rating_Score
0	Sheldon	Copper	42	9	25
1	Raj	Koothrappali	38	7	25
2	Leonard	Hofstadter	36	8	49
3	Howard	Wolowitz	41	8	62
4	Amy	Fowler	35	5	70

2. From List of List/Dictionaries

List of List:

```
import pandas as pd
l = [ ['Sheldon', 'Copper', 42, 9, 25],
      ['Raj', 'Koothrappali', 38, 7, 25],
      ['Leonard', 'Hofstadter', 36, 8, 49],
      ['Howard', 'Wolowitz', 41, 8, 62],
      ['Amy', 'Fowler', 35, 5, 70] ]
df = pd.DataFrame(l)
print(df)
```

0	Sheldon	Copper	42	9	25
1	Raj	Koothrappali	38	7	25
2	Leonard	Hofstadter	36	8	49
3	Howard	Wolowitz	41	8	62
4	Amy	Fowler	35	5	70

List of Dictionary:

```
import pandas as pd
l = [ {'first_name': 'Sheldon', 'last_name': 'Copper', 'age': 42,
      'Comedy_Score': 9, 'Rating_Score': 25},
      {'first_name': 'Raj', 'last_name': 'Koothrappali', 'age': 38, 'Comedy_Score':
      7, 'Rating_Score': 25},
      {'first_name': 'Leonard', 'last_name': 'Hofstadter', 'age': 36,
      'Comedy_Score': 8, 'Rating_Score': 49},
      {'first_name': 'Howard', 'last_name': 'Wolowitz', 'age': 41, 'Comedy_Score':
      8, 'Rating_Score': 62},
      {'first_name': 'Amy', 'last_name': 'Fowler', 'age': 35, 'Comedy_Score': 5,
      'Rating_Score': 70} ]
df = pd.DataFrame(l) print(df)
```

Output

	first_name	last_name	age	Comedy_Score	Rating_Score
0	Sheldon	Copper	42	9	25
1	Raj	Koothrappali	38	7	25
2	Leonard	Hofstadter	36	8	49
3	Howard	Wolowitz	41	8	62
4	Amy	Fowler	35	5	70

3. Text/CSV files

file: data.csv -----

first_name,last_name,age,Comedy_Score,Rating_Score

Sheldon, Copper, 42, 9, 25

Raj, Koothrappali, 38, 7, 25

Leonard, Hofstadter, 36, 8, 49

Howard, Wolowitz, 41, 8, 62

Amy, Fowler, 35, 5, 70

```
import pandas as pd
```

```
df = pd.read_csv("data.csv")
```

you can give a .txt file also, but the data should be comma separated print(df)

Output

	first_name	last_name	age	Comedy_Score	Rating_Score
0	Sheldon	Copper	42	9	25
1	Raj	Koothrappali	38	7	25
2	Leonard	Hofstadter	36	8	49
3	Howard	Wolowitz	41	8	62
4	Amy	Fowler	35	5	70

Note:- We will discuss details of read_csv() and to_csv() in *chapter 6 : Importing/Exporting Data between CSV files and Data Frames.*

index, columns, dtype parameter

```
import pandas as pd
```

```
l= [ ['Sheldon', 'Copper', 42, 9, 25],
      ['Raj', 'Koothrappali', 38, 7, 25],
      ['Leonard', 'Hofstadter', 36, 8, 49],
      ['Howard', 'Wolowitz', 41, 8, 62],
      ['Amy', 'Fowler', 35, 5, 70] ]
```

```
df = pd.DataFrame(l) print(df)
0 1 2 3 4 0 Sheldon    Copper 42 9 25
1 Raj Koothrappali 38 7 25
2 Leonard Hofstadter 36 8 49
3 Howard Wolowitz 41 8 62 4 Amy Fowler 35 5 70
df = pd.DataFrame(l, index =['P1','P2','P3','P4','P5'], columns =['first_name', 'last_name', 'age', 'Comedy_Score',
'Rating_Score'], dtype = float)
print(df)
Output
----first_name last_name age Comedy_Score Rating_Score
P1 Sheldon Copper 42.0 9.0 25.0
P2 Raj Koothrappali 38.0 7.0 25.0
P3 Leonard Hofstadter 36.0 8.0 49.0
P4 Howard Wolowitz 41.0 8.0 62.0
P5 Amy Fowler 35.0 5.0 70.0
```

Note:- To change the data types of already existing dataframe you can use **astype()** function.

```
df = df.astype ({'age': float})
```

3. DataFrame attributes

Attributes	Description
T	Transpose index and columns.
axes	Return a list representing the axes of the DataFrame.
columns	The column labels of the DataFrame.
index	The index (row labels) of the DataFrame.
dtypes	Return the dtypes in the DataFrame.
empty	Indicate whether DataFrame is empty.
at/iat	Access a single value for a row/column label pair.
loc/iloc	Access a group of rows and columns by label(s).
ndim	Return an int representing the number of axes / array dimensions.
shape	Return a tuple representing the dimensionality of the DataFrame.
size	Return an int representing the number of elements in this object.
values	Return a Numpy representation of the DataFrame.

Consider the following DataFrame **df**.

```
import pandas as pd
d = {
    "Roll_No" : [1,2,3],
    "Name" : ['Adrija','Baishali','Arijeet'],
    "Percentage" : [80,85,89]
} df = pd.DataFrame(d,index = ['STU-1','STU-2','STU-3']) print(df)
```

Output

Roll_No	Name	Percentage
STU-1	1 Adrija	80
STU-2	2 Baishali	85
STU-3	3 Arijeet	89

Let us see each attribute with an example on above dataframe **df**:

```
# ## 1. T df.T
"""
           STU-1  STU-2  STU-3
Roll_No      1      2      3
Name      Adrija Baishali Arijeet
Percentage  80      85      89
"""

# # 2. axes df.axes
"""
[Index(['STU-1', 'STU-2', 'STU-3'], dtype='object'),
 Index(['Roll_No', 'Name', 'Percentage'], dtype='object')]
"""

# # 3. columns df.columns
"""
Index(['Roll_No', 'Name', 'Percentage'], dtype='object') """

# ## 4. index df.index
"""
Index(['STU-1', 'STU-2', 'STU-3'], dtype='object') """

# # 5. dtypes df.dtypes
"""
```

```

Roll_No    int64
Name       object
Percentage  int64
dtype: object '''

# # 6.
empty
df.empty '''
False
'''

# ## 7. at
# ### at[INDEX,COL]
print(df.at['STU-2','Name'])
'''
Baishali
'''

# # 8. iat
print(df.iat[1,0])
'''
2
'''

# # 9. loc
# ## loc [start_Index : End_Index , start_col : End_col]
# Both the index elements will be included print(df.loc['STU-2:','Name':'Percentage'])
'''
      Name Percentage
STU-2 Baishali      85
STU-3 Arijeet      89
'''

# # 10. iloc
# ## iloc [start_Index : End_Index , start_col : End_col] print(df.iloc[0:2,1:2])
'''
      Name
'''

```

```

STU-1  Adrija
STU-2  Baishali
"""

# ## 11. ndim print(df.ndim)
""" 2
"""

# ## 12. shape print(df.shape)
"""
(3, 3)
"""

# ## 13. size df.size
""" 9
"""

# ## 14. values df.values """
array([[1, 'Adrija', 80],
       [2, 'Baishali', 85],
       [3, 'Arijeet', 89]], dtype=object)
"""

```

4. Operations on rows and columns - select, add, delete and rename

Consider the following DataFrame **df**:

```

import pandas as pd

d = {
    "Roll_No" : [1,2,3],
    "Name" : ['Adrija','Baishali','Arijeet'],
    "Percentage" : [80,85,89]
} df = pd.DataFrame(d,index = ['STU-1','STU-2','STU-3'])
print(df)
"""

```

	Roll_No	Name	Percentage
STU-1	1	Adrija	80
STU-2	2	Baishali	85
STU-3	3	Arijeet	89

```

"""

```

We will discuss each operation **Select, Add, Delete and Rename** on the above dataframe.

1. Select Columns and Rows from dataframe

Accessing the Columns

Syntax:

```
# DataFrame_Object [ Column_name ] #  
DataFrame_Object.Column_name
```

```
df['Roll_No'] STU-1  
1  
STU-2 2  
STU-3 3  
Name: Roll_No, dtype: int64 df[['Roll_No',  
'Name']]
```

	Roll_No	Name
STU-1	1	Adrija
STU-2	2	Baishali
STU-3	3	Arijeet

```
df.Roll_No STU-1  
1  
STU-2 2  
STU-3 3  
Name: Roll_No, dtype: int64
```

Note : - In the dot notation make sure not to put any quotation marks around the column name.

Accessing the Rows

.loc and **.iloc** attributes are used to access the row/slice from the dataframe.


```

Syntax:
# ## loc [start_Index : End_Index , start_col : End_col] # Both the index elements
will be included
df.loc['STU-1'] Roll_No
1
Name      Adrija
Percentage 80 Name: STU-1,
dtype: object
df.loc[['STU-1', 'STU-2']]
      Roll_No  Name Percentage STU-1
1  Adrija      80 STU-2      2 Baishali
85
df.loc['STU-1': 'STU-3']
      Roll_No  Name Percentage STU-1      1
Adrija      80
STU-2      2 Baishali      85
STU-3      3 Arijeet      89
# ## iloc [start_Index : End_Index , start_col : End_col]
df.iloc[0]
Roll_No      1
Name      Adrija
Percentage 80 Name: STU-
1, dtype: object
df.iloc[[0, 1]]
      Roll_No  Name Percentage STU-
1      1  Adrija      80 STU-2      2 Baishali
85
df.iloc[0: 3]
      Roll_No  Name Percentage STU-1      1
Adrija      80
STU-2      2 Baishali      85
STU-3      3 Arijeet      89

```

2. Adding Columns and Rows into dataframe

```

# ## Adding a Column
df['Adm_No'] = [1000,1001,1002]
print(df) Output
-----
      Roll_No  Name Percentage Adm_No
STU-1      1  Adrija      80  1000
STU-2      2 Baishali      85  1001
STU-3      3 Arijeet      89  1002
# ## Adding/Modifying a Row df.loc['STU-
4'] = [4, 'Mohan', 95] print(df) Output
-----
      Roll_No  Name Percentage STU-1      1
Adrija      80
STU-2      2 Baishali      85
STU-3      3 Arijeet      89
STU-4      4  Mohan      95

```

3. Deleting Columns and Rows from dataframe

```
# ## Deleting a Column del
df['Roll_No']
#OR df.pop('Roll_No')
#OR
df = df.drop('Roll_No',axis = 1)
print(df) Output
-----
      Name  Percentage STU-1
Adrija    80
STU-2 Baishali    85
STU-3 Arijeet    89
# ## Deleting a row
df = df.drop('STU-3',axis = 0) print(df)
Output
-----
      Roll_No  Name  Percentage STU-1
1  Adrija    80
STU-2  2 Baishali    85
```

4. Rename Columns and Rows of DataFrame

```
df = df.rename(columns = {'Roll_No':'New_RN', 'Name':'New_Name'}, index = {'STU-
1': 'S1', 'STU-3':'S3'})
print(df)

Output
-----
      New_RN New_Name  Percentage
S1      1  Adrija    80
STU-2    2 Baishali    85
S3      3  Arijeet    89
```

5. Basic DataFrame methods (head(), tail(), min(), max(), sum(), count(), mean())

Consider the following dataframe **df**:

```
import pandas as pd

data = {'first_name': ['Sheldon', 'Raj', 'Leonard', 'Howard', 'Amy'],
        'last_name': ['Copper', 'Koothrappali', 'Hofstadter', 'Wolowitz', 'Fowler'],
        'age': [42, 38, 36, 41, 35],
        'Comedy_Score': [9, 7, 8, 8, 5],
        'Rating_Score': [25, 25, 49, 62, 70]}

df = pd.DataFrame(data) print(df)
```

Output

	first_name	last_name	age	Comedy_Score	Rating_Score
0	Sheldon	Copper	42	9	25
1	Raj	Koothrappali	38	7	25
2	Leonard	Hofstadter	36	8	49
3	Howard	Wolowitz	41	8	62
4	Amy	Fowler	35	5	70

Let us now try all the methods/function on dataframe **df** head()

```
df.head() # by default 5 rows will come
```

	first_name	last_name	age	Comedy_Score	Rating_Score
0	Sheldon	Copper	42	9	25
1	Raj	Koothrappali	38	7	25
2	Leonard	Hofstadter	36	8	49
3	Howard	Wolowitz	41	8	62
4	Amy	Fowler	35	5	70

```
df.head(2)
```

	first_name	last_name	age	Comedy_Score	Rating_Score
0	Sheldon	Copper	42	9	25
1	Raj	Koothrappali	38	7	25

```
df.head(-2)
```

	first_name	last_name	age	Comedy_Score	Rating_Score
0	Sheldon	Copper	42	9	25
1	Raj	Koothrappali	38	7	25
2	Leonard	Hofstadter	36	8	49

tail()

```
df.tail() # by default 5 rows will come
  first_name  last_name  age  Comedy_Score  Rating_Score
0  Sheldon    Copper    42         9         25
1  Raj Koothrappali    38         7         25
2  Leonard   Hofstadter  36         8         49
3  Howard    Wolowitz   41         8         62 4  Amy
Fowler    35         5         70

df.tail(2) first_name last_name  age  Comedy_Score  Rating_Score
3  Howard    Wolowitz   41         8         62 4  Amy  Fowler
35         5         70

df.tail(-2) first_name  last_name  age  Comedy_Score  Rating_Score
2  Leonard   Hofstadter  36         8         49
3  Howard    Wolowitz   41         8         62
4  Amy      Fowler    35         5         70
```

min()

```
df['age'].min()
35

df.min() first_name  Amy last_name  Copper

age          35 Comedy_Score
5 Rating_Score    25 dtype:
object

df.min(axis=1)
0      9
1      7
2      8
3    8 4  5 dtype: int64
```

max()

```
df['age'].max() 42
```

sum()

```
df['age'].sum() 192
```

count()

```
df['age'].count() 5
```

mean()

```
df['age'].mean() 38.4
```

6. Importing/Exporting Data between CSV files and Data Frames.

read_csv()

This function is used to **create a DataFrame using CSV file**:

```
Syntax: pandas.read_csv(filepath_or_buffer, sep=',', delimiter=None,
header='infer', names=None, index_col=None, dtype=None)
```

Consider the following **csv file** :

File: Sample.csv

Roll_No,Name,Percentage

1,Adrija,80

2,Baishali,82

3,Arijeet,84

We will create our dataframe using the above csv file.

filepath or buffer parameter

```
import pandas as pd #Relative Path df
= pd.read_csv("sample.csv")
#OR df=pd.read_csv(filepath_or_buffer="sample.csv")
#Absolute Path df = pd.read_csv("C:\\Users\\XII IP\\Notes\\UNIT
1\\sample.csv") df
```

Output

	Roll_No	Name	Percentage
0	1	Adrija	80
1	2	Baishali	82
2	3	Arijeet	84

sep/delimiter Parameter

File: Sample.csv

Roll_No+Name+Percentage

1+Adrija+80

2+Baishali+82 3+Arijeet+84

```
df = pd.read_csv("sample.csv",sep="+")
```

#OR

```
df = pd.read_csv("sample.csv",delimiter="+")
```

Output

	Roll_No	Name	Percentage
0	1	Adrija	80
1	2	Baishali	82
2	3	Arijeet	84

header Parameter

```
df = pd.read_csv("sample.csv",header=1)
      1      Adrija 80 0 2 Baishali 82 1
3 Arijeet 84 df =
pd.read_csv("sample.csv",header=[0,2])
      Roll_No  Name Percentage
      2      Baishali      82 0      3
Arijeet      84
```

names Parameter

```
df = pd.read_csv("sample.csv",names=['A','B','C'])
```

```
      A      B      C 0 Roll_No
Name Percentage
1      1  Adrija      80
2      2 Baishali      82 3      3 Arijeet      84
```

```
df = pd.read_csv("sample.csv",names=['A','B','C'], header=0)
```

```
      A      B C 0 1
Adrija 80
1 2 Baishali 82
2 3 Arijeet 84
```

index_col Parameter

```
df = pd.read_csv("sample.csv",index_col=2)
```

```
      Roll_No  Name
Percentage
80      1  Adrija
82      2 Baishali
84      3 Arijeet
```

dtype Parameter

```
import numpy as np
df = pd.read_csv("sample.csv",dtype={"Percentage":np.float64})
```

```
      Roll_No  Name Percentage
0      1  Adrija      80.0
1      2 Baishali      82.0
2      3 Arijeet      84.0
```

to_csv()

This function is used to **create a CSV file from an existing DataFrame**. Consider the following DataFrame **df**:

```
import pandas as pd
d = {
    "Roll_No" : [1,2,3],
    "Name" : ['Adrija','Baishali','Arijeet'],
    "Percentage" : [80,85,89]
} df = pd.DataFrame(d,index = ['STU-1','STU-2','STU-3'])
print(df)
```

	Roll_No	Name	Percentage
STU-1	1	Adrija	80
STU-2	2	Baishali	85
STU-3	3	Arijeet	89

We will perform our operation if the above DataFrame **df**.

path_or_buf Parameter

```
# Relative Path df.to_csv(path_or_buf =
"stu_per.csv")
# OR df.to_csv("stu_per.csv")
# Absolute Path df.to_csv("C:\\Users\\Tanmay\\Desktop\\stu_per.csv")
# file created: stu_per.csv
,Roll_No,Name,Percentage
0,1,Adrija,80.0
1,2,Baishali,82.0
2,3,Arijeet,84.0
```

sep Parameter

```
df.to_csv("stu_per.csv",sep="#")
# File: stu_per.csv
#Roll_No#Name#Percentage
0#1#Adrija#80.0
1#2#Baishali#82.0
2#3#Arijeet#84.0
```

columns Parameter

```
df.to_csv("stu_per.csv",columns=["Name"])
# File: stu_per.csv
,Name
0,Adrija
1,Baishali 2,Arijeet
```

header Parameter

```
df.to_csv("stu_per.csv",header = False)
# File: stu_per.csv
0,1,Adrija,80.0
1,2,Baishali,82.0
2,3,Arijeet,84.0
```

index Parameter

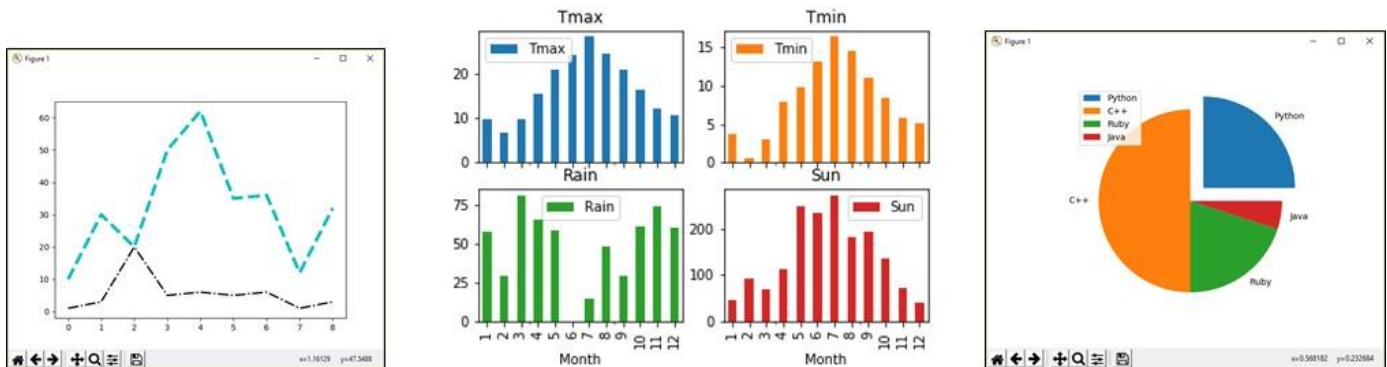
```
df.to_csv("stu_per.csv",index = False,header = False)  
# File: stu_per.csv  
1,Adrija,80.0  
2,Baishali,82.0  
3,Arijeet,84.0
```

Data Visualization

1. Introduction to Data Visualization
2. Line Plot
3. Bar Graph
4. Histogram
5. Customization of Plots
6. Most Important Questions and Answers

1. Introduction

Data Visualization:- It is the **graphical representation of information and data**. By using visual elements like **charts, graphs, and maps**, data visualization tools provide an accessible way to **see and understand trends, outliers, and patterns in data**.



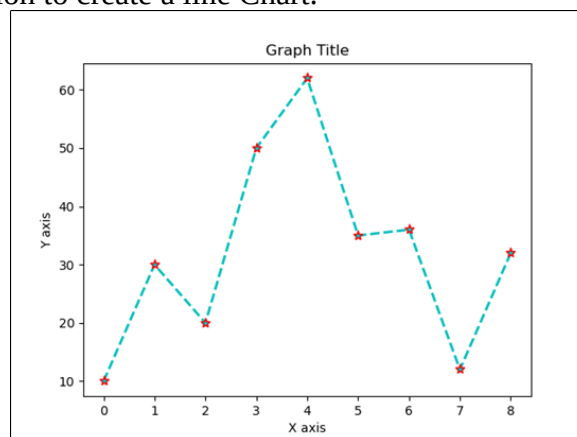
To draw the graphs/plots in python we have to import the **pyplot interface** of matplotlib library.

```
import matplotlib.pyplot as plt
```

pyplot has various methods/functions that are used to draw the graphs.

2. Line Chart

- A **line chart** or **Line graph** is a type of chart which displays information as a series of data points called '**markers**' connected by **straight line segments**.
- Pyplot has a **plot()** function to create a line Chart.



Syntax:

```
<matplotlib.pyplot>.plot(X-Axis, Y-Axis, color, linewidth, linestyle or ls, marker, markersize, markeredgecolor)
```

- **X-Axis, Y-Axis:** Points on X & Y Axis in the form of Sequence
- **color :** Color of Line. Should be valid color identified by Pyplot
- **linewidth :** Width of line in points (*float value*)
- **linestyle or ls :** Style of the line (Ex:- **solid** or '-' [*Default*], **dashed** or '--', **dashdot** or '-.', **dotted** or ':')

- **marker** : A symbol denoting the marker style such as 'o', '*', '+' etc.
- **markersize** : Marker size in points (float value)
- **markeredgecolor** : Color of marker. Should be valid color identified by Pyplot **Note:-** Colors Identified by Pyplot:

Character	Color		Character	Color		Character	Color
b	blue		m	magenta		c	cyan
g	green		y	yellow		w	white
r	red		k	black			

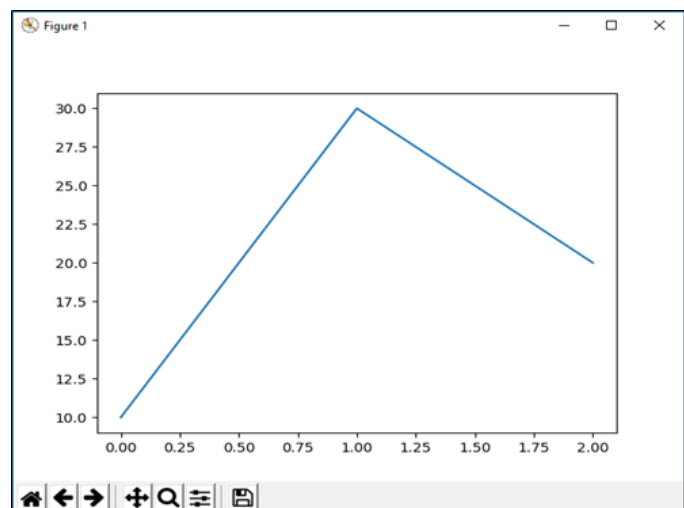
With only Y – Axis Points

If we give only one list, it will take the x axis by-default [0,1,2,3,.....].

```
# importing Pyplot import matplotlib.pyplot
as plt

# Giving Y-Axis Points plt.plot([10,30,20])

# Displaying the Plot plt.show()
```



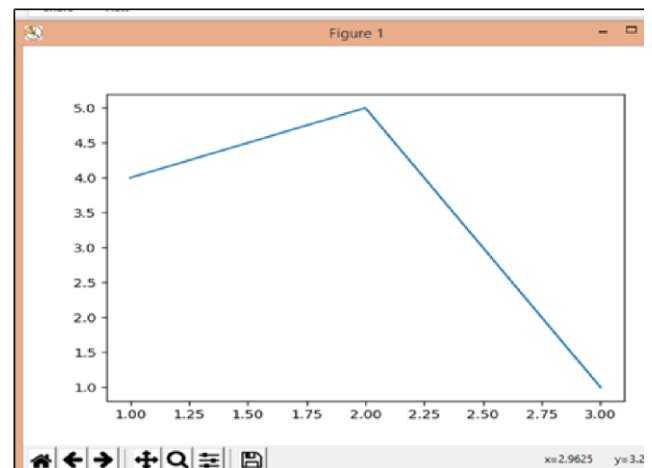
With X & Y – Axis Points

In this case we have to provide both x-axis and y-axis points in the form of a list.

```
# Importing pyplot import matplotlib.pyplot
as plt

# Plotting to our canvas
# the graph will be generated for # points
((1,4),(2,5),(3,1)) plt.plot([1,2,3],[4,5,1])

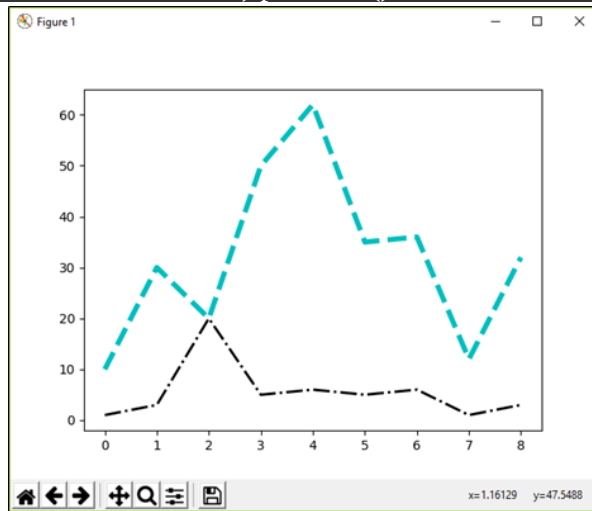
# Displaying what we plotted plt.show()
```



Changing Line Width , Line Color and Line Style

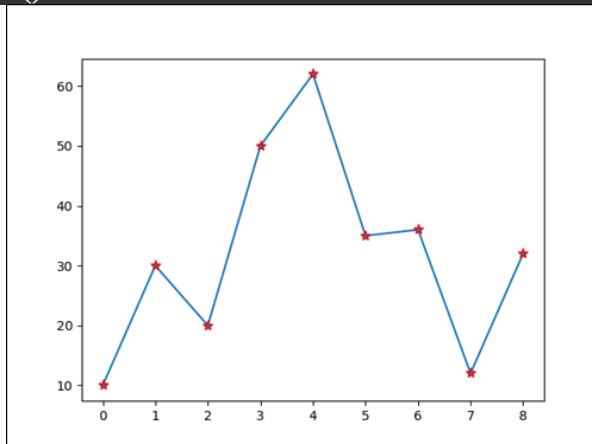
linewidth, color and linestyle or ls parameters of plot() function are used to change these properties.

```
import matplotlib.pyplot as plt
arr=[10,30,20,50,62,35,36,12,32]
arr1=[1,3,20,5,6,5,6,1,3]
plt.plot(arr, color='c', linewidth=4.0, ls='dashed')
plt.plot(arr1, color='k', linewidth=2.0, ls='dashdot')
plt.show()
```



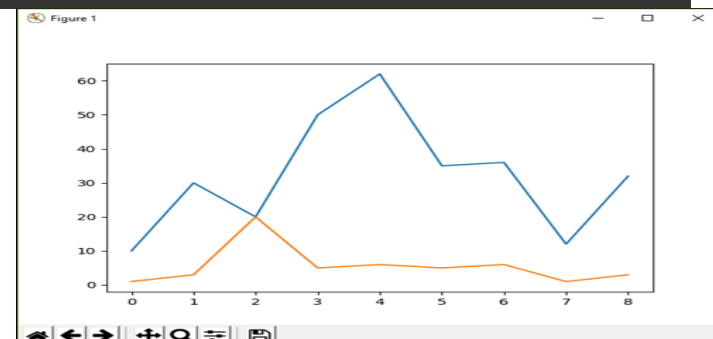
Changing Marker Type, Marker Size and Marker Color

```
import matplotlib.pyplot as plt
arr=[10,30,20,50,62,35,36,12,32]
plt.plot(arr, marker='*', markersize=8.0,
markedgecolor='red')
plt.show()
```



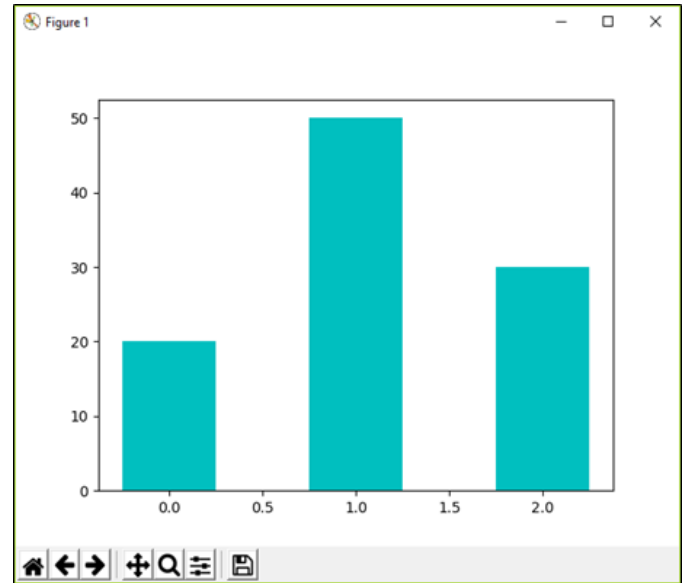
Plotting Two or More Graphs on same plot

```
import matplotlib.pyplot as plt
arr=[10,30,20,50,62,35,36,12,32]
arr1=[1,3,20,5,6,5,6,1,3]
plt.plot(arr)
plt.plot(arr1)
plt.show()
```



3. Bar Graph

- A diagram in which the **numerical values** of variables are represented by the **height or length of lines or rectangles**.
- For the bar chart we will use the **bar() function**, where we define the position of the bars on the X axis and their height on the Y axis.
- Additionally, we can also configure other characteristics for the chart, like width of the bars, color, among others.
- **bar()** and **barh()** functions are used for **vertical and horizontal bar Graph** creation respectively.



bar() function

Syntax:

```
<matplotlib.pyplot>.bar(<X-Axis>, <Y-Axis>, width, color, align)
```

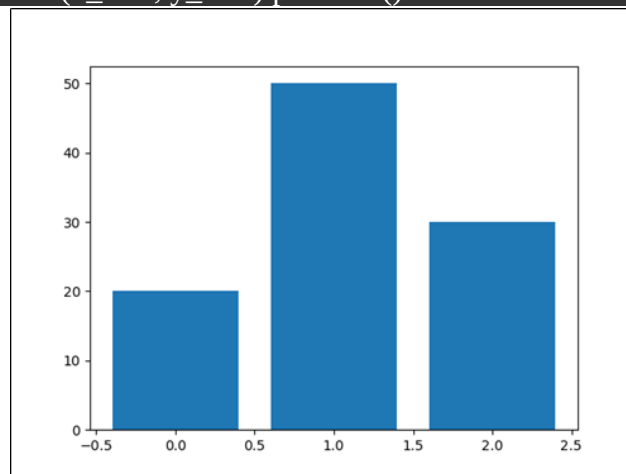
- **X-Axis** : Points on X Axis in the form of Sequence
- **Y-Axis (height)** : Height or Points on Y Axis in the form of Sequence
- **color** : Color of Bar. Should be valid color identified by Pyplot
- **width** : Width of bars in the form of scalar value or as a sequence
- **align** : {‘center’, ‘edge’}, optional, default ‘center’

Note: X-Axis and Y-Axis should have the same number of values and both the values are required.

Creating a first bar chart

```
import matplotlib.pyplot as plt;

# Variables for the bar chart
# Both X and Y arrays are required and both should be of same size y_axis =
[20,50,30] x_axis = [0,1,2] plt.bar(x_axis, y_axis) plt.show()
```

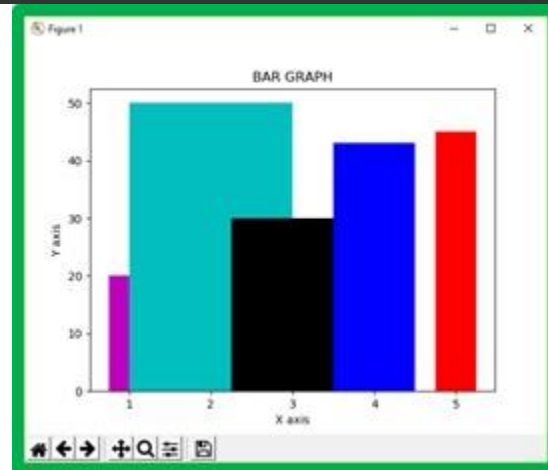
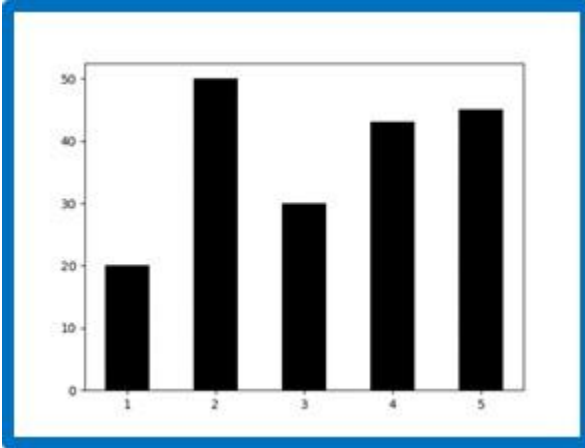


Color and Width of Bar

```
import matplotlib.pyplot as plt
arr1 = [1,2,3,4,5]
arr2 = [20,50,30,43,45]

# changing color and width with scalar value
plt.bar(arr1, arr2, width=0.5, color='k')

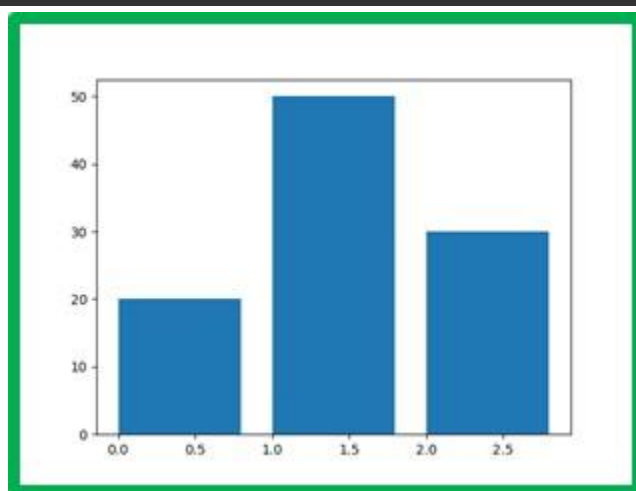
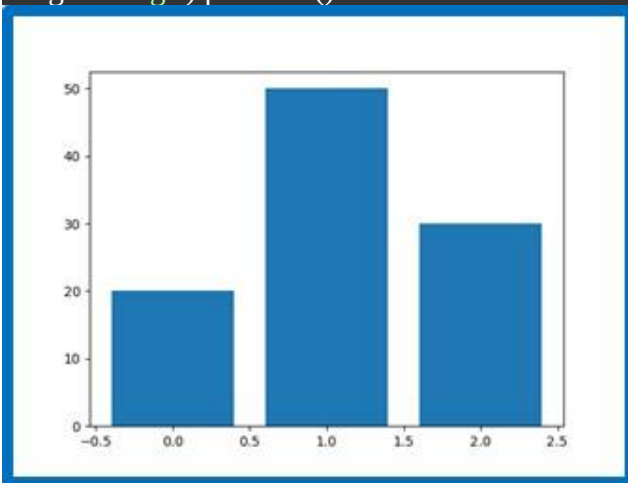
# changing color and width with sequence of values
plt.bar(arr1, arr2, width=[.5, 2, 1.5, 1, 0.5], color=['m', 'c', 'k', 'b', 'r'])
plt.show()
```



Alignment of Bar

```
import matplotlib.pyplot as plt
y_axis = [20, 50, 30]
x_axis = [0, 1, 2]

plt.bar(x_axis, y_axis, align='center')
plt.bar(x_axis, y_axis, align='edge')
plt.show()
```



barh() function syntax

Syntax:

```
barh(<y_axis>, <x_axis>, height, color, align)
```

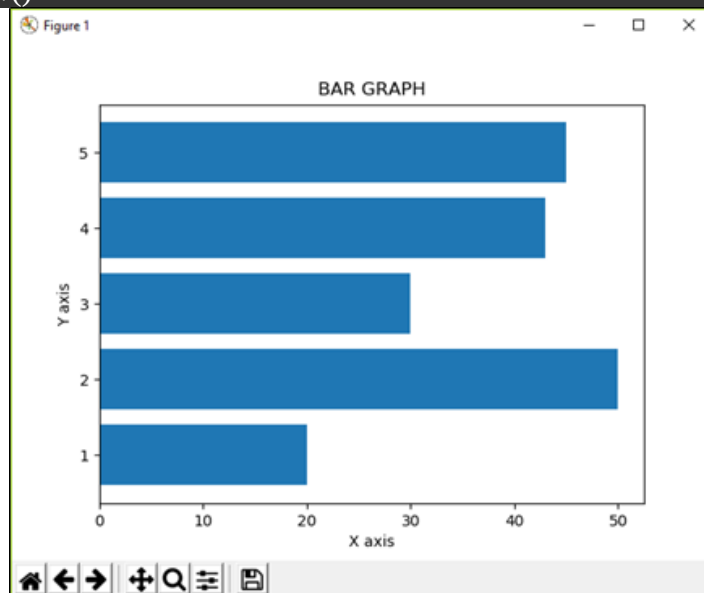
- **Y-Axis** : Points on Y Axis in the form of Sequence
- **X-Axis (width)** : height or Points on X Axis in the form of Sequence
- **color** : Color of Bar. Should be valid color identified by Pyplot
- **height** : Width of bars in the form of scalar value or as a sequence
- **align** : {'center', 'edge'}, optional, default 'center'

Note: X-Axis and Y-Axis should have the same number of values and both the values are required.

Horizontal Bar chart (barh())

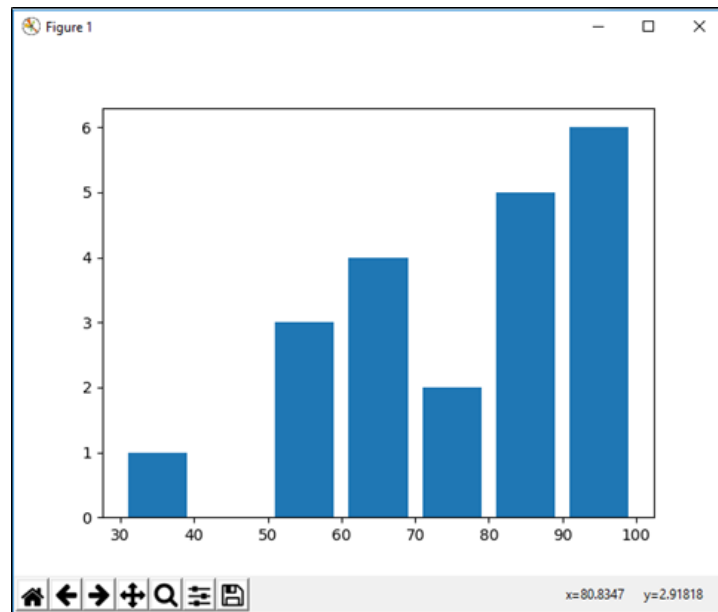
```
import matplotlib.pyplot as plt
```

```
arr1 = [1,2,3,4,5] arr2 =  
[20,50,30,43,45]  
plt.barh(arr1,arr2) plt.show()
```

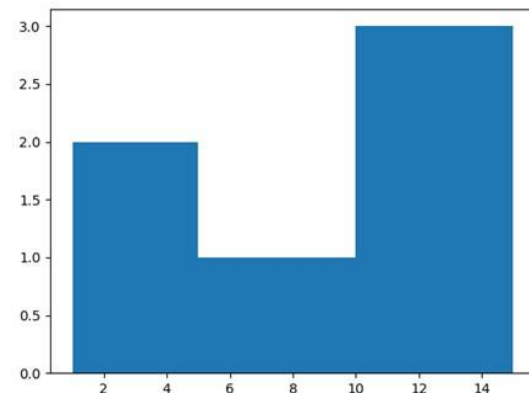
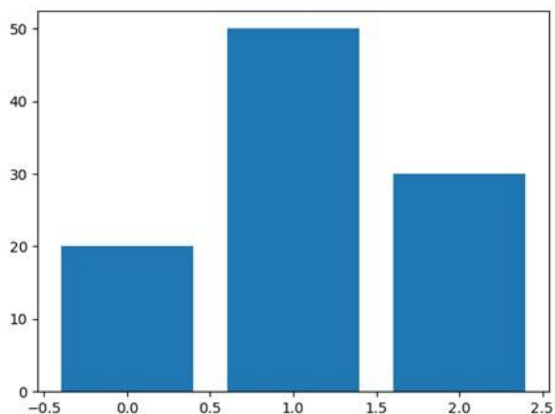


4. Histogram

- A **histogram** is a **graphical display of data using bars of different heights**.
- In a histogram, **each bar groups numbers into ranges**. Taller bars show that more data falls in that range.
- **hist() function** is used for generating histogram in Pyplot.

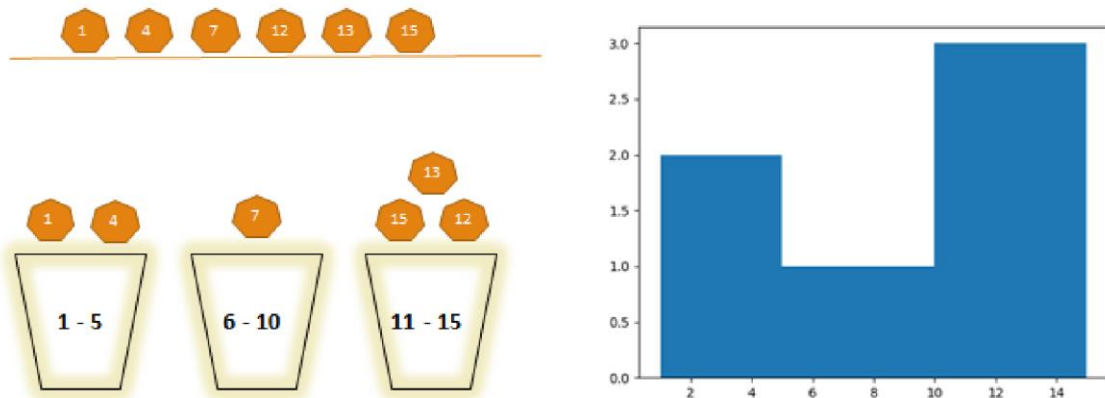


Difference between Bar Graph vs Histogram



- **Histograms** are used to show **distributions of variables** while **bar charts** are used to **compare variables**.
- **Histograms** plot quantitative data with ranges of the **data grouped into bins or intervals** while **bar charts** plot **categorical data**.

Histogram



hist() function

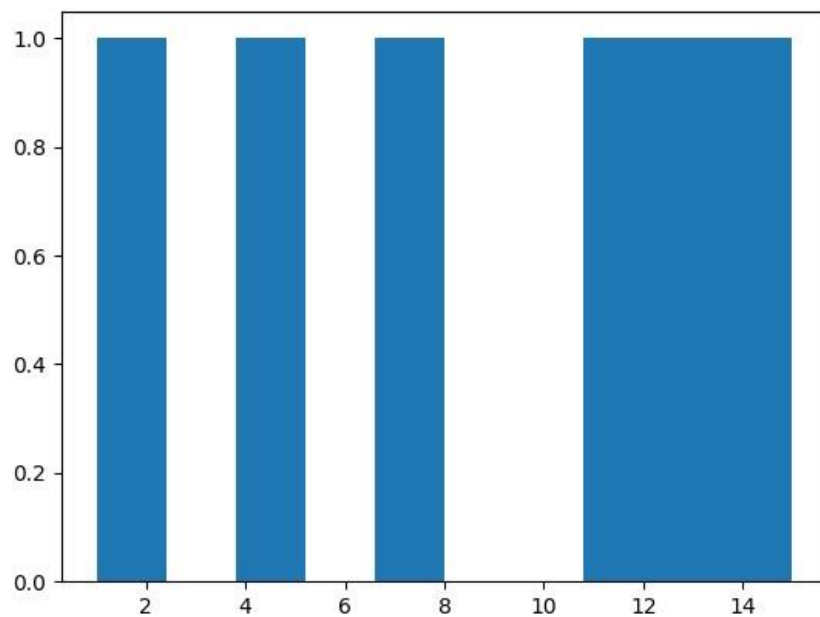
Syntax:

```
<matplotlib.pyplot>.hist(x, bins = None, cumulative = False, histtype =  
'bar', align = 'mid', orientation = 'vertical')
```

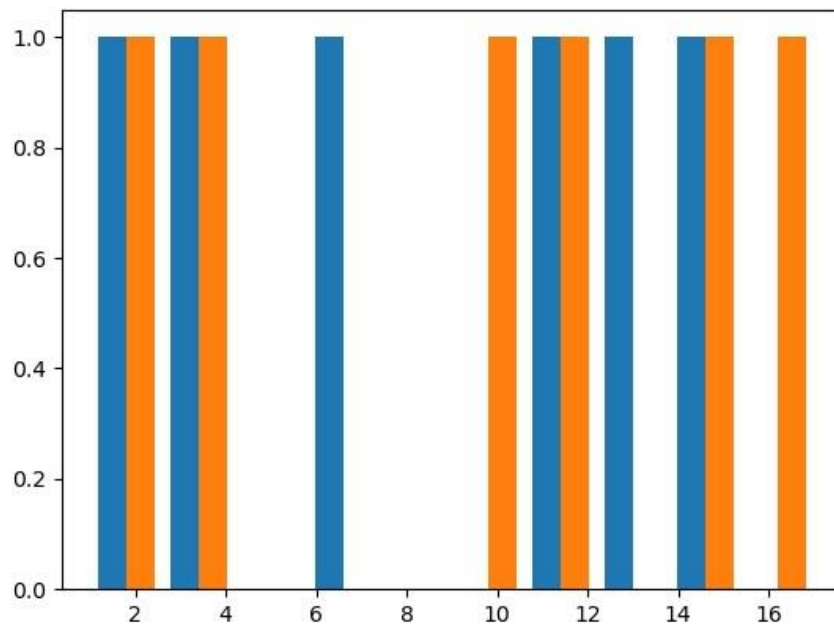
- **X** : Array or sequence of arrays to be plotted on Histogram
- **bins** : an integer or sequence, used to divide the range in the histogram
- **cumulative** : True or False, Default is False
- **histtype** : {'bar', 'barstacked', 'step', 'stepfilled'}, Default 'bar'
- **align** : {'left', 'mid', 'right'}, default is 'mid'
- **orientation** : {'horizontal', 'vertical'}, Default 'vertical'

x Parameter (data)

```
import matplotlib.pyplot as plt  
b = [1,4,7,12,13,15]  
plt.hist(x = b) plt.show()
```

```
import matplotlib.pyplot as plt
a = [1,4,7,12,13,15]
b = [10,14,17,2,3,11]
plt.hist([a,b])
plt.show()
```

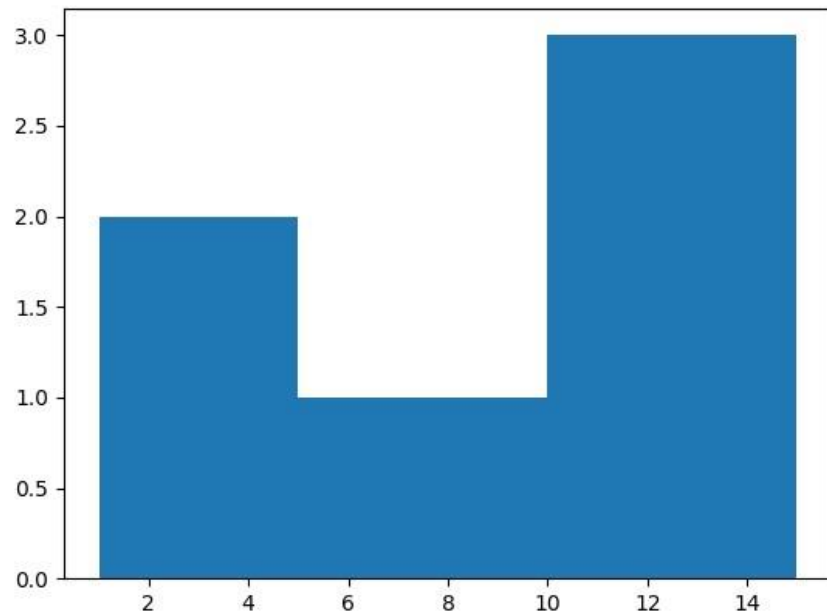


bins Parameter (data)

By **default** the **bins will be divided automatically**. Python checks the lowest and highest values from the given numbers and divides them into equal size bins.

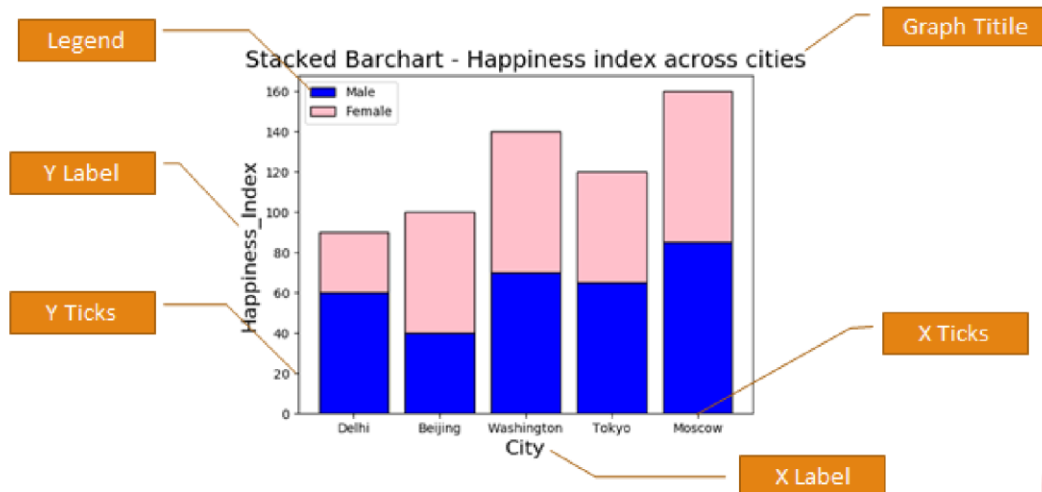
```
import matplotlib.pyplot as plt
```

```
n = [1,4,7,12,13,15] plt.hist(n, bins=[1,5,10,15]) plt.show()
```



5. Customization of plots

Anatomy of a pyplot graph



X & Y Labels for the plot

```
import matplotlib.pyplot as plt
```

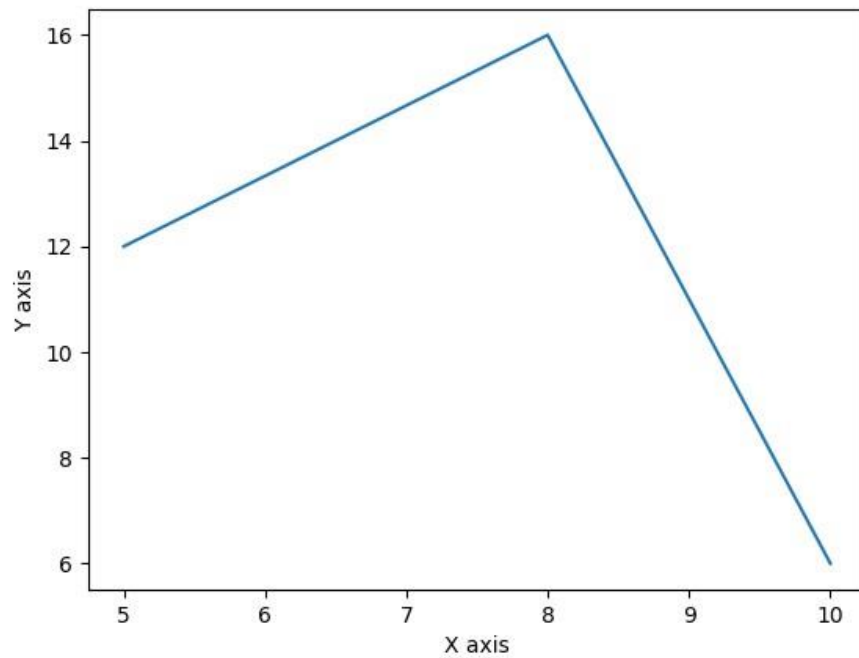
```
x = [5,8,10] y =
```

```
[12,16,6]
```

```
plt.plot(x,y)
```

```
plt.ylabel('Y axis') plt.xlabel('X axis')
```

```
plt.show()
```



Title for the plot

```
import matplotlib.pyplot as plt
```

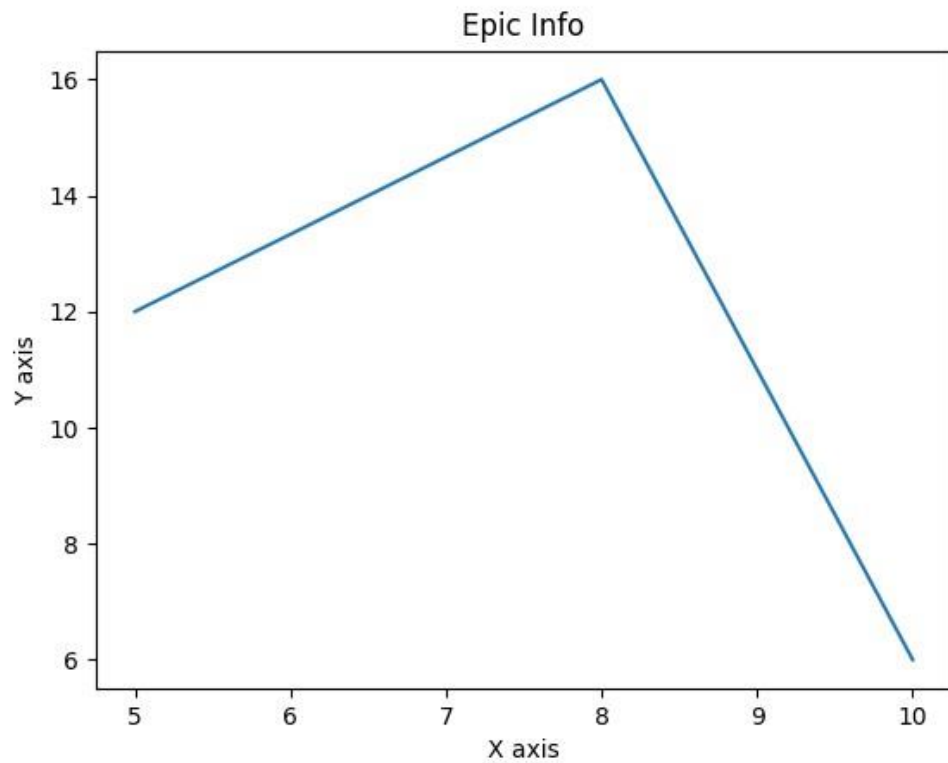
```
x = [5,8,10] y =
```

```
[12,16,6]
```

```
plt.plot(x,y)
```

```
plt.title('Epic Info') plt.ylabel('Y axis')
```

```
plt.xlabel('X axis') plt.show()
```



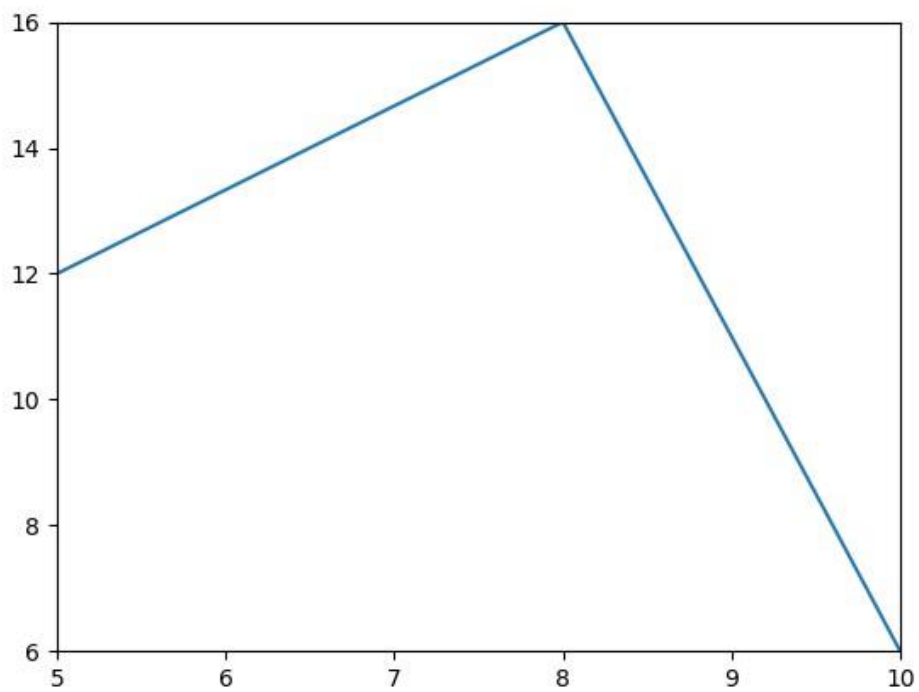
X & Y Limit for the plot

```
import matplotlib.pyplot as plt
```

```
x = [5,8,10] y =  
[12,16,6]  
plt.plot(x,y)
```

```
plt.ylim(6, 16) plt.xlim(5, 10)
```

```
plt.show()
```



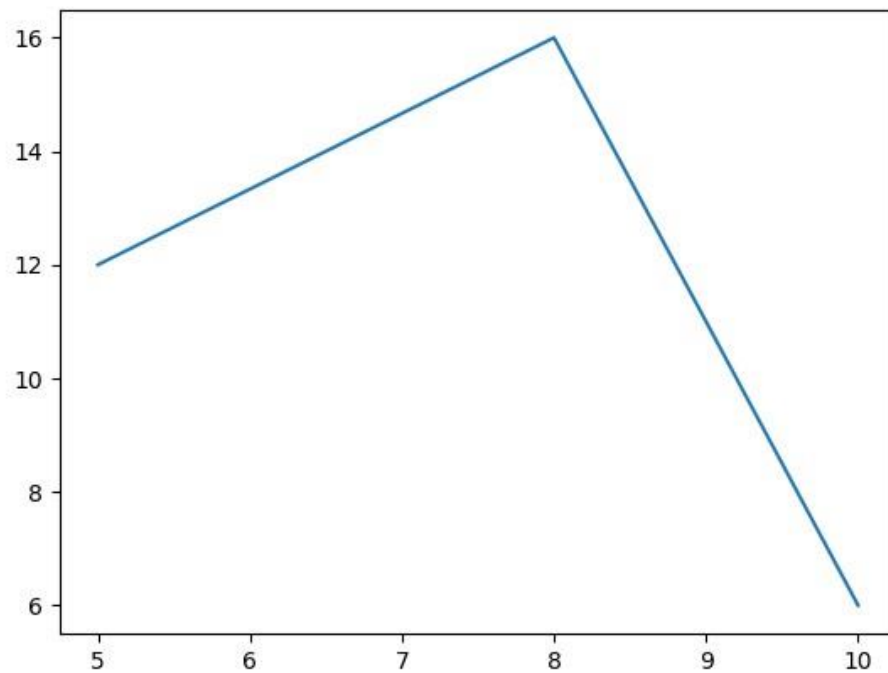
X & Y Ticks for the plot

```
import matplotlib.pyplot as plt

x = [5,8,10] y =
[12,16,6]
plt.plot(x,y)

plt.xticks([5,6,7,8,9,10]) plt.yticks([6,8,10,12,14,16])

plt.show()
```



Adding Legend in the plot

Syntax:

```
<matplotlib.pyplot>.legend(loc="upper right")
```

loc:

"upper right" [Default]

"upper left"

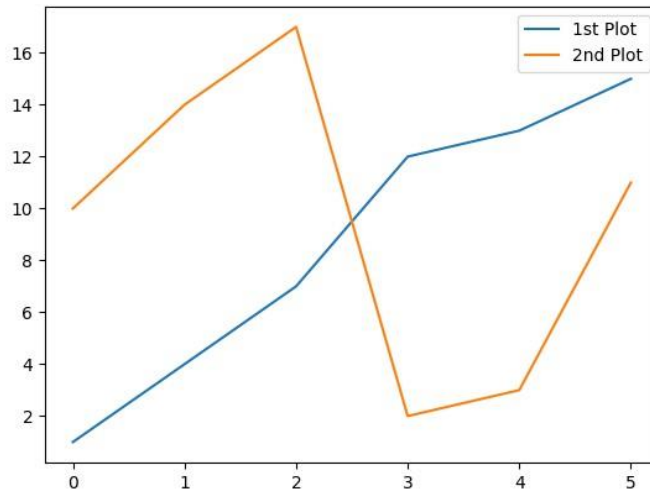
"lower right"

"lower left"

Note: For legends you have to give label parameters in plot(), bar() or hist() functions.

Example:

```
import matplotlib.pyplot as plt  
a = [1,4,7,12,13,15] b = [10,14,17,2,3,11]  
plt.plot(a, label = "1st Plot")  
plt.plot(b, label = "2nd Plot")  
plt.legend()  
plt.show()
```



Saving the plot

Syntax:

```
<matplotlib.pyplot>.savefig(<string with filename and Path>)
```

Note: If only the file name given, save the fig in the current directory. You can save figure in Popular Formats like (.pdf, .png, .eps etc)

```
import matplotlib.pyplot as plt
a = [1,4,7,12,13,15]
b = [10,14,17,2,3,11]
plt.plot(a)
plt.plot(b)
plt.savefig("C:\\Users\\acer\\Desktop\\Delete\\dv.png")
plt.show()
```

dv.png files saved on the desktop.

This PC > Desktop				
Name	Date modified	Type	Size	
dv.png	17-01-2021 13:49	PNG File	25 KB	
abc.py	17-01-2021 13:49	Python File	1 KB	
myModule.py	12-01-2021 09:37	Python File	3 KB	
abc.txt	15-12-2020 11:44	Text Document	1 KB	

1. Math functions

- POWER () or POW()
- ROUND ()
- MOD ().

2. Text functions

- UCASE ()/UPPER ()
- LCASE ()/LOWER ()
- MID ()/SUBSTRING ()/SUBSTR ()
- LENGTH ()

- e. LEFT (), RIGHT ()
- f. INSTR ()
- g. LTRIM (), RTRIM (), TRIM ()

3. Date Functions

- a. NOW ()
- b. DATE ()
- c. MONTH ()
- d. MONTHNAME ()
- e. YEAR ()
- f. DAY ()
- g. DAYNAME ()

4. Most Important Questions and Answers

1. Math Functions

Consider the following table **student**:

```
mysql> select * from student;
```

Roll_No	Name	Class	DOB	Gender	City	Marks
1	Nanda	X	1995-06-06	M	Agra	551
2	Saurabh	XII	1993-05-07	M	Mumbai	462
3	Sanal	XI	1994-05-06	F	Delhi	400
4	Trisla	XII	1995-08-08	F	Mumbai	450
5	Store	XII	1995-10-08	M	Delhi	369
6	Marisla	XI	1994-12-12	F	Dubai	250
7	Neha	X	1995-12-08	F	Moscow	377
8	Nishant	X	1995-06-12	M	Moscow	489

We will perform all our operations on this table.

a. POWER() or POW()

Syntax:

```
POWER( m, n ) OR POW( m, n)
```

Parameters or Arguments:

- **m:** Numeric value. It is the base used in the calculation.
- **n:** Numeric value. It is the exponent used in the calculation.

Example:


```
mysql> SELECT POW(3,2);
+-----+
| POW(3,2) |
+-----+
|          9 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT POWER(3,-2);
+-----+
| POWER(3,-2) |
+-----+
| 0.1111111111111111 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT POW(marks,2) FROM student;
+-----+
| POW(marks,2) |
+-----+
|      303601 |
|      213444 |
|      160000 |
|      202500 |
|      136161 |
|       62500 |
|      142129 |
|      239121 |
+-----+
8 rows in set (0.00 sec)
```

b. ROUND()

Syntax:

```
ROUND( number, [ decimal_places ] )
```

Parameters or Arguments:

- **number:** The number to round.
- **decimal_places:** The number of decimal places to round. This value must be a positive or negative integer. If this parameter is omitted, the ROUND function will round the number to 0 decimal places.

Example:

```
mysql> SELECT ROUND(12.5);
```

```
+-----+
```

```
| ROUND(12.5) |
```

```
+-----+
```

```
|          13 |
```

```
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> SELECT ROUND(12.4);
```

```
+-----+
```

```
| ROUND(12.4) |
```

```
+-----+
```

```
|          12 |
```

```
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> SELECT ROUND(12.465,1);
```

```
+-----+
```

```
| ROUND(12.465,1) |
```

```
+-----+
```

```
|          12.5 |
```

```
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> SELECT ROUND(12.465,-1);
```

```
+-----+
```

```
| ROUND(12.465,-1) |
```

```
+-----+
```

```
|          10 |
```

```

+-----+
1 row in set (0.00 sec)

mysql> SELECT ROUND(150,-2);
+-----+
| ROUND(150,-2) |
+-----+
|          200 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT ROUND(149,-2);
+-----+
| ROUND(149,-2) |
+-----+
|          100 |
+-----+
1 row in set (0.00 sec)

```

c. MOD()

Syntax:

```
MOD( n, m ) OR n MOD m OR n % m
```

Parameters or Arguments:

- **n**: Number. ●
- m**: Divisor.

Example:

```

mysql> SELECT MOD(5,2);
+-----+
| MOD(5,2) |
+-----+

```

```

| 1 |
+-----+
1 row in set (0.01 sec) mysql>
SELECT 5 MOD 2;
+-----+
| 5 MOD 2 |
+-----+
| 1 |
+-----+
1 row in set (0.00 sec)
mysql> SELECT 5%2;
+-----+
| 5%2 |
+-----+
| 1 |
+-----+
1 row in set (0.00 sec)

```

2. Text functions

a. UCASE ()/UPPER ()

It converts all characters in the specified string to uppercase. If there are characters in the string that are not letters, they are unaffected by this function.

Syntax:

```
UCASE( string )
```

Parameter or Arguments:

- **string:** The string to convert to uppercase.

Example:

```
mysql> SELECT UCASE("cbsecsip");
+-----+
| UCASE("cbsecsip") |
+-----+
| CBSECSIP          |
+-----+
1 row in set (0.01 sec)

mysql> SELECT UPPER("cbsecsip");
+-----+
| UPPER("cbsecsip") |
+-----+
| CBSECSIP          |
+-----+
1 row in set (0.00 sec)

mysql> SELECT UPPER(name) FROM student;
+-----+
| UPPER(name) |
+-----+
| NANDA      |
| SAURABH    |
| SANAL      |
| TRISLA     |
| STORE      |
| MARISLA    |
| NEHA       |
| NISHANT    |
+-----+
8 rows in set (0.01 sec)
```

b. LCASE ()/LOWER ()

It converts all characters in the specified string to lowercase. If there are characters in the string that are not letters, they are unaffected by this function.

Syntax:

```
LCASE( string )
```

Parameter or Arguments:

- **string:** The string to convert to lowercase.

Example:

```
mysql> SELECT LCASE("CBSECSIP");
+-----+
| LCASE("CBSECSIP") |
+-----+
| cbsecsip          |
+-----+ 1 row in set
(0.00 sec)

mysql> SELECT LOWER("CBSECSIP");
+-----+

| LOWER("CBSECSIP") |
+-----+
| cbsecsip          |
+-----+
1 row in set (0.00 sec) mysql> SELECT LOWER(name)
FROM student;
+-----+
| LOWER(name) |
+-----+
| nanda      |
| saurabh    |
| sanal      |
| trisla     |
| store      |
| marisla    |
| neha       |
| nishant    |
+-----+
8 rows in set (0.00 sec)
```

c. MID ()/SUBSTRING ()/SUBSTR ()

This function allows you to extract a substring from a string.

Syntax:

```
MID( string, start_position, [length] )
      SUBSTR( string, start_position, [ length ] )
      SUBSTRING( string, start_position, [ length ] )
```

Parameter or Arguments:

- **string:** The string from which Substring is required.
- **start_position:** The position to begin extraction. The first position in the string is always 1.
- **Length:** The number of characters to extract **Example:**

```
mysql> SELECT MID("ABCDEFGH",2);
+-----+
| MID("ABCDEFGH",2) |
+-----+
| BCDEFGH          |
+-----+
1 row in set (0.00 sec)

mysql> SELECT MID("ABCDEFGH",2,3);
+-----+
| MID("ABCDEFGH",2,3) |
+-----+
| BCD                 |
+-----+
1 row in set (0.00 sec)
```

d. LENGTH()

It function returns the length of the specified string (measured in bytes).

Syntax:

```
LENGTH( string )
```

Parameter or Arguments:

- **string:** The string to return the length for.

Example:

```
mysql> SELECT LENGTH("ABCDEFGH");
```

```

+-----+
| LENGTH("ABCDEFG") |
+-----+
|                7 |
+-----+
1 row in set (0.03 sec)

```

e. LEFT ()

LEFT function allows you to extract a substring from a string, starting from the left-most character.

Syntax:

```
LEFT( string, number_of_characters )
```

Parameter or Arguments:

- **string:** The string that you wish to extract from.
- **number_of_characters:** The number of characters that you wish to extract from string starting from the left-most character.

Example:

```

mysql> SELECT LEFT("ABCDEFG",2);
+-----+
| LEFT("ABCDEFG",2) |
+-----+
| AB                |
+-----+
1 row in set (0.00 sec)

```

f. RIGHT ()

RIGHT function allows you to extract a substring from a string, starting from the right-most character.

Syntax:

```
RIGHT( string, number_of_characters )
```

Parameter or Arguments:

- **string:** The string that you wish to extract from.
- **number_of_characters:** The number of characters that you wish to extract from string starting from the right-most character.

Example:

```
mysql> SELECT RIGHT("ABCDEFGF",2);
+-----+
| RIGHT("ABCDEFGF",2) |
+-----+
| FG                  |
+-----+
1 row in set (0.00 sec)
```

g. INSTR()

INSTR function returns the location of a substring in a string.

Syntax:

```
INSTR( string, substring )
```

Parameter or Arguments:

- **string:** The string to search.
- **substring:** The substring to search for in string.

Example:

```
mysql> SELECT INSTR("LEARN CS AND IP FROM CBSECSIP","CBSECSIP");
+-----+
| INSTR("LEARN CS AND IP FROM CBSECSIP","CBSECSIP") |
+-----+
|                                                    22 |
+-----+
1 row in set (0.01 sec)
```

h. LTRIM()

LTRIM function removes all space characters from the left-hand side of a string.

Syntax:

```
LTRIM( string )
```

Parameter or Arguments:

- **string:** The string to trim the space characters from the left-hand side.

Example:

```
mysql> SELECT LTRIM("          CBSECSIP          ");
+-----+
| LTRIM("          CBSECSIP          ") |
+-----+
| CBSECSIP                             |
+-----+
1 row in set (0.02 sec)
```

i. RTRIM()

RTRIM function removes all space characters from the right-hand side of a string.

Syntax:

```
RTRIM( string )
```

Parameter or Arguments:

- **string:** The string to trim the space characters from the right-hand side.

Example:

```
mysql> SELECT RTRIM("          CBSECSIP          ");
+-----+
| RTRIM("          CBSECSIP          ") |
+-----+
|          CBSECSIP                     |
+-----+
1 row in set (0.01 sec)
```

j. TRIM()

TRIM function removes all specified characters either from the beginning or the end of a string.

Syntax:

```
TRIM([ LEADING | TRAILING | BOTH ] [ trim_character FROM ] string)
```

Parameter or Arguments:

- **LEADING:** Optional. Removes the trim_character from the front of the string.
- **TRAILING:** Optional. Removes the trim_character from the end of string.

- **BOTH:** Optional. Removes the trim_character from the front and end of string.
- **trim_character:** Optional. The character that will be removed from string. If this parameter is omitted, it will remove space characters from string.
- **string:** The string to trim.

Example:

```
mysql> SELECT TRIM("          CBSECSIP          ");
+-----+
| TRIM("          CBSECSIP          ") |
+-----+
| CBSECSIP                             |
+-----+
1 row in set (0.00 sec)
```

3. Date Functions

a. NOW()

NOW function returns the current date and time.

Syntax:

```
NOW()
```

Example:

```
mysql> SELECT NOW();
+-----+
| NOW() |
+-----+
| 2021-01-18 13:54:14 |
+-----+
1 row in set (0.00 sec)
```

b. DATE()

DATE function extracts the date value from a date or datetime expression.

Syntax:

```
DATE( expression )
```

Parameter or Arguments:

- **expression:** The date or datetime value from which the date should be extracted.

Example:

```
mysql> SELECT DATE('2021-01-18 13:54:14');
+-----+
| DATE('2021-01-18 13:54:14') |
+-----+
| 2021-01-18 |
+-----+
1 row in set (0.02 sec)
```

c. MONTH ()

MONTH function returns the month portion of a date value.

Syntax:

```
MONTH( date_value )
```

Parameter or Arguments:

- **date_value:** A date or datetime value from which to extract the month.

Example:

```
mysql> SELECT MONTH('2021-01-18 13:54:14');
+-----+
| MONTH('2021-01-18 13:54:14') |
+-----+
|                               1 |
+-----+
1 row in set (0.00 sec)
```

d. MONTHNAME ()

MONTHNAME function returns the full name of the month for a date.

Syntax:

```
MONTHNAME( date_value )
```

Parameter or Arguments:

- **date_value:** A date or datetime value from which to extract the full month name.

Example:

```
mysql> SELECT MONTHNAME('2021-01-18 13:54:14');
+-----+
| MONTHNAME('2021-01-18 13:54:14') |
+-----+
| January                          |
+-----+
1 row in set (0.02 sec)
```

e. YEAR ()

YEAR function returns the year portion of a date value.

Syntax:

```
YEAR( date_value )
```

Parameter or Arguments:

- **date_value:** A date or datetime value from which to extract the year.

Example:

```
mysql> SELECT YEAR('2021-01-18 13:54:14');
+-----+
| YEAR('2021-01-18 13:54:14') |
+-----+
|                2021         |
+-----+
1 row in set (0.00 sec)
```

f. DAY ()

DAY function returns the day portion of a date value.

Syntax:

```
DAY( date_value )
```

Parameter or Arguments:

- **date_value:** The date or datetime value from which to extract the day.

Example:

```
mysql> SELECT DAY('2021-01-18 13:54:14');
+-----+
| DAY('2021-01-18 13:54:14') |
+-----+
|                18         |
+-----+
```

g. DAYNAME ()

DAYNAME function returns the weekday name for a date.

Syntax:

```
DAYNAME( date_value )
```

Parameter or Arguments:

- **date_value:** The date or datetime value from which to extract the weekday name.

Example:

```
mysql> SELECT DAYNAME('2021-01-18 13:54:14');
+-----+
| DAYNAME('2021-01-18 13:54:14') |
+-----+
| Monday                          |
+-----+
```

1. Aggregate Functions
 - a. MAX ()
 - b. MIN ()
 - c. AVG ()
 - d. SUM ()
 - e. COUNT ()
2. Difference between COUNT() and COUNT (*)
3. GROUP BY and HAVING
4. Difference between WHERE and HAVING clause
5. ORDER BY
6. Most Important Questions and Answers

1. Aggregate Functions

An aggregate function **performs a calculation on multiple values** and **returns a single value**.

1. MAX ()
2. MIN ()
3. AVG ()
4. SUM ()
5. COUNT ()

City Table

City Name	PIN Code	Population (in Cr)
Delhi	110092	1.9
Mumbai	400001	1.84
Kolkata	700001	1.49
Hyderabad	500001	0.68
Bangalore	560001	0.84

} SUM()

↓

Sum of Population = 6.75

Consider following table **student**, we will perform all of our operations on this table.

```
mysql> select * from student;
```

Roll_No	Name	Class	DOB	Gender	City	Marks
1	Nanda	X	1995-06-06	M	Agra	551
2	Saurabh	XII	1993-05-07	M	Mumbai	462
3	Sanal	XI	1994-05-06	F	Delhi	400
4	Trisla	XII	1995-08-08	F	Mumbai	450
5	Store	XII	1995-10-08	M	Delhi	369
6	Marisla	XI	1994-12-12	F	Dubai	250
7	Neha	X	1995-12-08	F	Moscow	377
8	Nishant	X	1995-06-12	M	Moscow	489

1. MAX ()

MAX function returns the maximum value of an expression.

Syntax:

```
SELECT MAX(aggregate_expression)
FROM tables
[WHERE conditions];
```

Parameter or Arguments:

- **aggregate_expression:** This is the column or expression from which the maximum value will be returned.

Example:


```
mysql> SELECT MAX(marks) FROM student;
+-----+
| MAX(marks) |
+-----+
|      551 |
+-----+
1 row in set (0.04 sec)
```

2. MIN ()

MIN function returns the minimum value of an expression.

Syntax:

```
SELECT MIN(aggregate_expression)
FROM tables
[WHERE conditions];
```

Parameter or Arguments:

- **aggregate_expression:** This is the column or expression from which the minimum value will be returned.

Example:

```
mysql> SELECT MIN(marks) FROM student;
```

```
+-----+
| MIN(marks) |
+-----+
|      250 |
+-----+
1 row in set (0.00 sec)
```

3. AVG ()

AVG function returns the average value of an expression.

Syntax:

```
SELECT AVG(aggregate_expression)
FROM tables
[WHERE conditions];
```

Parameter or Arguments:

- **aggregate_expression:** This is the column or expression that will be averaged.

Example:

```
mysql> SELECT AVG(marks) FROM student;
+-----+
| AVG(marks) |
+-----+
| 418.5000 |
+-----+
1 row in set (0.02 sec)
```

4. SUM ()

SUM function returns the summed value of an expression.

Syntax:

```
SELECT SUM(aggregate_expression)
FROM tables
[WHERE conditions];
```

Parameter or Arguments:

- **aggregate_expression:** This is the column or expression that will be summed.

Example:

```
mysql> SELECT SUM(marks) FROM student;
+-----+
| SUM(marks) |
+-----+
| 3348 |
+-----+
1 row in set (0.00 sec)
```

5. COUNT ()

COUNT function returns the count of an expression.

Syntax:

```
SELECT COUNT(aggregate_expression)
FROM tables
[WHERE conditions];
```

Parameter or Arguments:

- **aggregate_expression:** This is the column or expression whose non-null values will be counted.

Example:

```
mysql> SELECT COUNT(marks) FROM student;
```

```
+-----+
| COUNT(marks) |
+-----+
|           8 |
+-----+
1 row in set (0.00 sec)
```

2. Difference between COUNT() and COUNT (*)

The **count(*)** returns **all rows** whether column contains null value or not while **count(columnName)** returns the number of rows **except null rows**.

```
mysql> SELECT * FROM employee;
+-----+-----+
| name    | salary |
+-----+-----+
| RAM     | 5000   |
| MOHAN   | 4000   |
| SHYAM   | NULL   |
| PRATIMA | 8000   |
| KARAN   | 7000   |
+-----+-----+ 5 rows in set (0.00 sec) mysql>
SELECT COUNT(*) FROM employee;
+-----+
| COUNT(*) |
+-----+
|    5    |
+-----+
1 row in set (0.16 sec) mysql> SELECT COUNT(salary)
FROM employee;
+-----+
| COUNT(salary) |
+-----+
|    4    |
+-----+
1 row in set (0.00 sec)
```

3. Group by and Having

- The **GROUP BY** clause **combines all those records** (Rows) that have **identical values in a particular field** (Column) or a group of fields.
- This grouping results into one summary record per group.

- The GROUP BY clause is **used with Aggregate Functions**.

Syntax:

```
SELECT <COL_NAME>
FROM <TABLE>
WHERE <CONDITION>
GROUP BY <COL_NAME>
HAVING <CONDITION ON GROUPS>;
```

Example:

```
SELECT SUM(salary) FROM employee;
```

Emp Name	DOJ	Dep_ID	Salary
Jhon	2000	D1	50000
David	2001	D1	26000
Robert	2000	D1	60000
Alax	2000	D2	40000
Ivan	2000	D2	60000
Anderson	2001	D2	55000

Agg Fun: SUM(Salary)

SUM(Salary)
291000

```
SELECT dep_id, SUM(salary) FROM employee GROUP BY dep_id;
```

Emp Name	DOJ	Dep_ID	Salary
Jhon	2000	D1	50000
David	2001	D1	26000
Robert	2000	D1	60000
Alax	2000	D2	40000
Ivan	2000	D2	60000
Anderson	2001	D2	55000

SUM(Salary) with **GROUP BY Dep_ID**

Dep_ID	SUM(Salary)
D1	136000
D2	155000

Nested Grouping: using more than one column

- If you want to **GROUP BY** using more than one column, you can **give** all columns by **using comma separation**.
- The GROUP BY clause is used with Aggregate Functions.
- **All the fields (Columns) other than used in aggregate function should be put in GROUP BY clause.**

Syntax:

```
SELECT Col1, Col2, AGG_FUN(Col3)
FROM Tab1
[WHERE <Condition>]
GROUP BY Col1, Col2 ;
```

Example:

```
SELECT dep_id, SUM(salary) FROM employee GROUP BY dep_id;
SELECT dep_id, doj, SUM(salary) FROM employee GROUP BY dep_id, doj;
```

Emp Name	DOJ	Dep_ID	Salary
Jhon	2000	D1	50000
David	2001	D1	26000
Robert	2000	D1	60000
Alax	2000	D2	40000
Ivan	2000	D2	60000
Anderson	2001	D2	55000

**SUM(Salary) with
GROUP BY Dep_ID**

Dep_ID	SUM(Salary)
D1	136000
D2	155000

**SUM(Salary) with
GROUP BY Dep_ID, DOJ**

DOJ	Dep_ID	SUM(Salary)
2000	D1	111000
2001	D1	26000
2000	D2	100000
2001	D2	55000

HAVING

- Used to give **Conditions on Groups**.

Syntax:

```
SELECT <COL_NAME>
FROM <TABLE>
WHERE <CONDITION>
GROUP BY <COL_NAME>
HAVING <CONDITION ON GROUPS>;
```

Example:

```
mysql> select * from student;
```

Roll_No	Name	Class	DOB	Gender	City	Marks
---------	------	-------	-----	--------	------	-------

1	Nanda	X	1995-06-06	M	Agra	551
2	Saurabh	XII	1993-05-07	M	Mumbai	462
3	Sanal	XI	1994-05-06	F	Delhi	400
4	Trisla	XII	1995-08-08	F	Mumbai	450
5	Store	XII	1995-10-08	M	Delhi	369
6	Marisla	XI	1994-12-12	F	Dubai	250
7	Neha	X	1995-12-08	F	Moscow	377
8	Nishant	X	1995-06-12	M	Moscow	489

8 rows in set (0.00 sec)

```
mysql> SELECT class, COUNT(*)
```

```
-> FROM student
```

```
-> GROUP BY class
```

```
-> HAVING count(*)>2;
```

class	COUNT(*)
-------	----------

X	3
XII	3

2 rows in set (0.12 sec)

5. Difference between WHERE and HAVING

- **Conditions on normal Columns** can be placed using **WHERE statement**.
- The **HAVING clause** places **conditions on groups** in contrast to WHERE clause that places conditions on individual rows.
- While **WHERE conditions cannot include aggregate functions**, **HAVING conditions can do so**.
- You can also place more than one condition in a HAVING clause.

6. Order by

The SQL **ORDER BY** clause is used to **sort the records in the result set** for a SELECT statement.

Syntax:

```
SELECT expressions FROM
tables
[WHERE conditions]
[GROUP BY <COL_NAME>]
HAVING <CONDITION ON GROUPS>]
ORDER BY COL1,COL2,.. [ ASC | DESC ];
```

- **ASC** – Ascending order by COLUMNS [Default]
- **DESC** – Descending order by COLUMNS **Example:**

```
mysql> select * from student;
```

Roll_No	Name	Class	DOB	Gender	City	Marks
1	Nanda	X	1995-06-06	M	Agra	551
2	Saurabh	XII	1993-05-07	M	Mumbai	462
3	Sanal	XI	1994-05-06	F	Delhi	400
4	Trisla	XII	1995-08-08	F	Mumbai	450
5	Store	XII	1995-10-08	M	Delhi	369
6	Marisla	XI	1994-12-12	F	Dubai	250
7	Neha	X	1995-12-08	F	Moscow	377
8	Nishant	X	1995-06-12	M	Moscow	489

8 rows in set (0.00 sec)

```
mysql> SELECT * FROM student ORDER BY marks;
```

Roll_No	Name	Class	DOB	Gender	City	Marks
---------	------	-------	-----	--------	------	-------

```
+-----+-----+-----+-----+-----+-----+
| 6 | Marisla | XI   | 1994-12-12 | F   | Dubai | 250 |
| 5 | Store   | XII  | 1995-10-08 | M   | Delhi | 369 |
| 7 | Neha    | X    | 1995-12-08 | F   | Moscow | 377 |
| 3 | Sanal   | XI   | 1994-05-06 | F   | Delhi | 400 |
| 4 | Trisla  | XII  | 1995-08-08 | F   | Mumbai | 450 |
| 2 | Saurabh | XII  | 1993-05-07 | M   | Mumbai | 462 |
| 8 | Nishant | X    | 1995-06-12 | M   | Moscow | 489 |
| 1 | Nanda   | X    | 1995-06-06 | M   | Agra   | 551 |
+-----+-----+-----+-----+-----+-----+

```

8 rows in set (0.00 sec) mysql> **SELECT * FROM student ORDER BY**
city, marks;

```
+-----+-----+-----+-----+-----+-----+
| Roll_No | Name      | Class | DOB          | Gender | City    | Marks |
+-----+-----+-----+-----+-----+-----+
| 1 | Nanda   | X     | 1995-06-06 | M     | Agra    | 551 |
| 5 | Store   | XII   | 1995-10-08 | M     | Delhi   | 369 |
| 3 | Sanal   | XI    | 1994-05-06 | F     | Delhi   | 400 |
| 6 | Marisla | XI    | 1994-12-12 | F     | Dubai   | 250 |
| 7 | Neha    | X     | 1995-12-08 | F     | Moscow  | 377 |
| 8 | Nishant | X     | 1995-06-12 | M     | Moscow  | 489 |
| 4 | Trisla  | XII   | 1995-08-08 | F     | Mumbai  | 450 |
| 2 | Saurabh | XII   | 1993-05-07 | M     | Mumbai  | 462 |
+-----+-----+-----+-----+-----+-----+

```

8 rows in set (0.01 sec) mysql> **SELECT * FROM student ORDER BY city desc,**
marks;

```
+-----+-----+-----+-----+-----+-----+
| Roll_No | Name      | Class | DOB          | Gender | City    | Marks |
+-----+-----+-----+-----+-----+-----+
| 4 | Trisla  | XII   | 1995-08-08 | F     | Mumbai  | 450 |
| 2 | Saurabh | XII   | 1993-05-07 | M     | Mumbai  | 462 |
| 7 | Neha    | X     | 1995-12-08 | F     | Moscow  | 377 |

```

```
| 8 | Nishant | X     | 1995-06-12 | M     | Moscow  | 489 |
| 6 | Marisla | XI    | 1994-12-12 | F     | Dubai   | 250 |
| 5 | Store   | XII   | 1995-10-08 | M     | Delhi   | 369 |
| 3 | Sanal   | XI    | 1994-05-06 | F     | Delhi   | 400 |
| 1 | Nanda   | X     | 1995-06-06 | M     | Agra    | 551 |
+-----+-----+-----+-----+-----+-----+

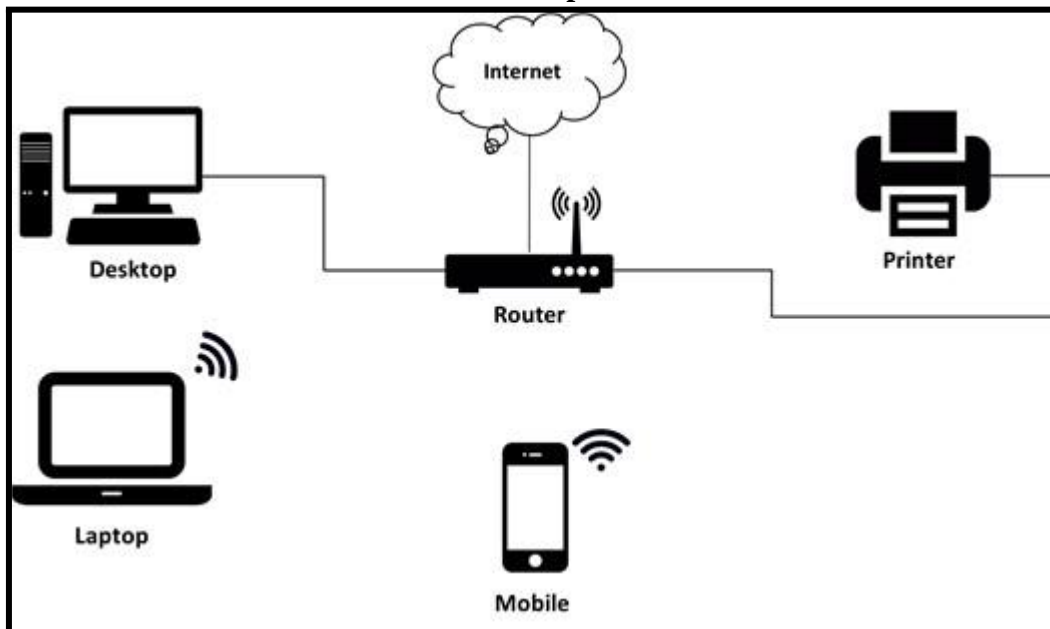
```

8 rows in set (0.00 sec)

- ■ ■
- 1. Introduction to networks
- 2. Types of network
 - a. PAN
 - b. LAN
 - c. MAN
 - d. WAN
- 3. Network Devices
 - a. Modem
 - b. Hub
 - c. Switch
 - d. Repeater
 - e. Router
 - f. Gateway
- 4. Network Topologies
 - a. Star
 - b. Bus
 - c. Tree
 - d. Mesh
- 5. Most Important Questions and Answers

1. Introduction to networks

When **two or more autonomous computing devices connected to one another** in order to exchange information or share resources, form a **Computer Network**.



Major Components:

- **Host:** It refers to a computer that are attached to a network
- **Servers:** The master computer is called Server. It facilitates the sharing of data, software (s/w) and hardware (h/w) .
- **Client:** It is a host computer that sends requests to the server for some services.
- **Network-Hardware:** Specialized hardware is required to carry out various roles in a network For eg. Establishing connections, controlling network traffic and many more.
- **Communication Channel:** MediumNodes on a network interact with each other through various communication channels. It can be :
 - **WIRED:** Twisted pair cable,coaxial cable, optical fiber
 - **WIRELESS:** Microwave, Radiowave, Satellite, Infrared, Laser etc.
- **Software:** Software comprises the Network protocols (set of rules governing the Network) and Network OS etc. **Example** Of Network Protocols-http, ftp, SMTP, TCP/IP etc.
- **Network Services:** These are the other software applications required in a network for different functionalities for g. VOIP, DNS etc.

Advantages of Computer Network

- Resource Sharing
- Cost Reduction
- Collaborative User Interaction
- Share Storage
- Fast Communication

Disadvantages of Computer Network

- Security Concerns
- Lack of Robustness
- Needs an Efficient Handler
- Lack of Independence
- It requires an expensive set-up.

2. Types of network

Mainly there are 4 types of networks:

- PAN (Personal area Network)
- LAN (Local Area Network)
- MAN (Metropolitan Area Network)
- WAN (Wide Area Network)

a. PAN (Personal Area Network)

Personal Area Network is a network arranged within an individual person, typically within a range of **10 meters (30 feet)**.

Types of Personal Area Network (PAN) :

- **Wireless PAN** – Wireless Personal Area Network (WPAN) is connected through signals such as infrared, ZigBee, Bluetooth and ultra-wideband etc.
- **Wired PAN** – Wired PAN is connected through cables/wires such as Firewire or USB (Universal Serial Bus).

b. LAN (Local Area Network)

Local Area Network is a group of computers connected to each other in a **small area such as a building, office.**

- It covers an area of a few kilometer radius (**approx. 1-10 km**).
- It is **managed by a single person / Organization.**

c. MAN (Metropolitan Area Network)

A metropolitan area network is a network that covers a **larger geographic area by interconnecting a different LAN** to form a larger network e.g. **within a city.**

- Covers an area of a **few kilometers to few hundred kilometers.**
- **Owned by Organization or Government.**
- Network of **schools, banks, government offices** within a city are examples of MAN.

d. WAN (Wide Area Network)

A Wide Area Network is a network that extends over a **large geographical area such as states or countries.**

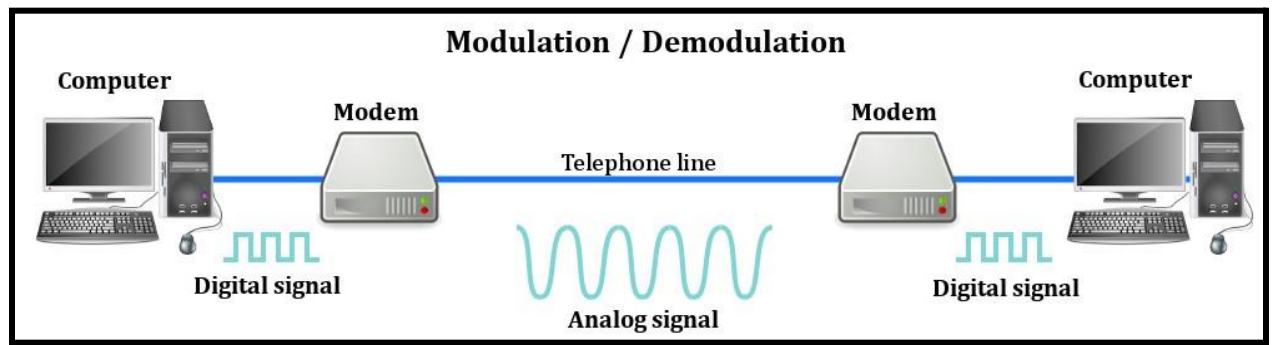
- Covers over **hundreds of kilometer radius.**
- It is usually formed by **interconnecting LANs, MANs or may be other WANs.**
- Network of **ATMs, BANKs, National or International organization offices spread over a country , continent** are examples of WAN.
- The **internet is one of the biggest WAN in the world.**

3. Network Devices

a. Modem

Modem is an abbreviation for **Modulator – Demodulator.**

Modulator converts information from **digital mode to analog** mode at the transmitting end and **demodulator** converts the same from **analog to digital** at receiving end.



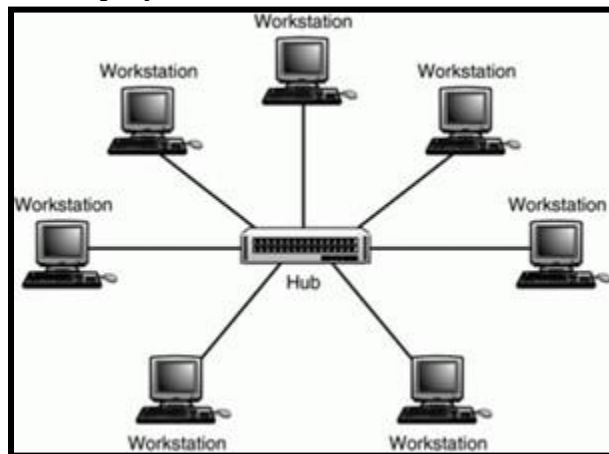
b. Hub

Hub is an electronic device that **connects several nodes to form a network** and **redirect the received information to all the nodes** in a broadcast mode. It is called a **non-intelligent device**.

It is mostly **used in Star or Ring topology** and **works on MAC addresses**.

Types:

- **Active hub:** electrically amplify the signal.
- **Passive hub:** do not amplify the data.

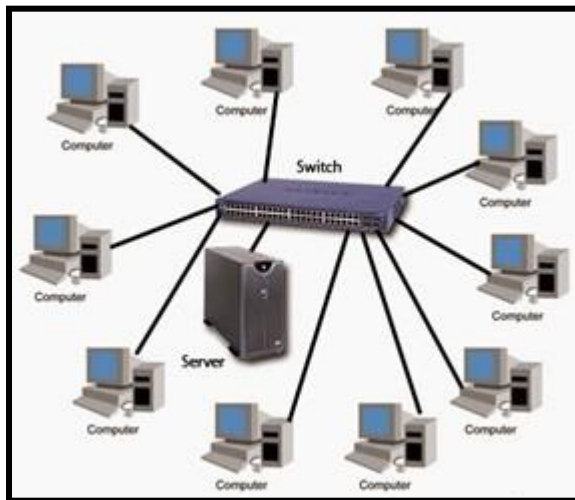


c. Switch

It is an **intelligent device** that connects several nodes to form a network and **redirect the received information only to the intended node(s)**.

A switch is also **called Intelligent HUB**.

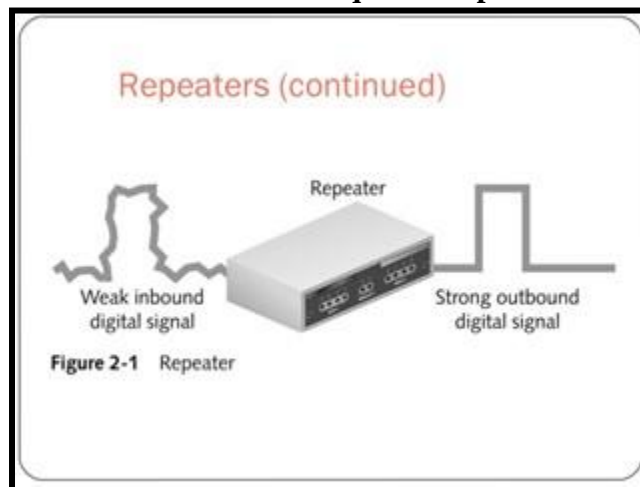
Switch **stores the MAC address table of all the connected nodes**, that helps it to send the data to the desired node.



d. Repeater

It is a device which **accepts the weak signals and amplifies the signals for further transmission.**

- Different types of wires have different transmission distances, hence repeaters should be used after according to the types of wire.
- Generally we consider that **most wires require a repeater after 100m distance.**

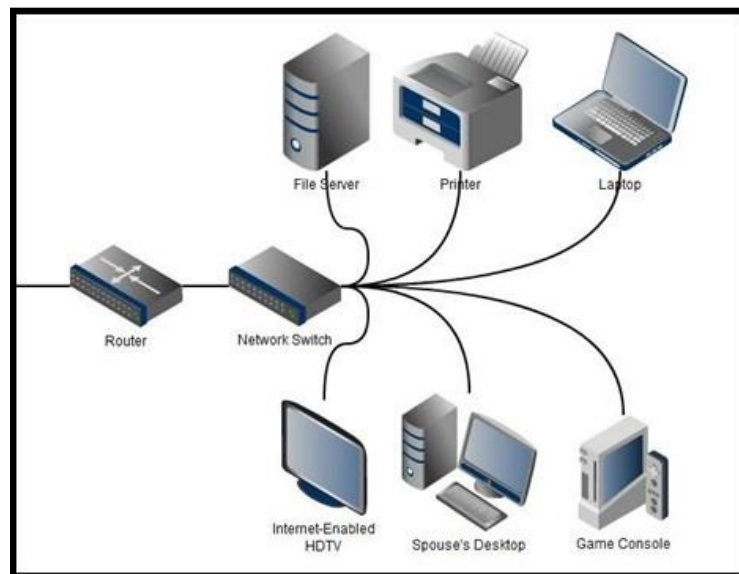


e. Router

Router **works on IP addresses.**

Routers normally connect **LANs and WANs together.**

It has a dynamically updating routing table (stores IP Addresses) based on which they make decisions on routing the data packets.

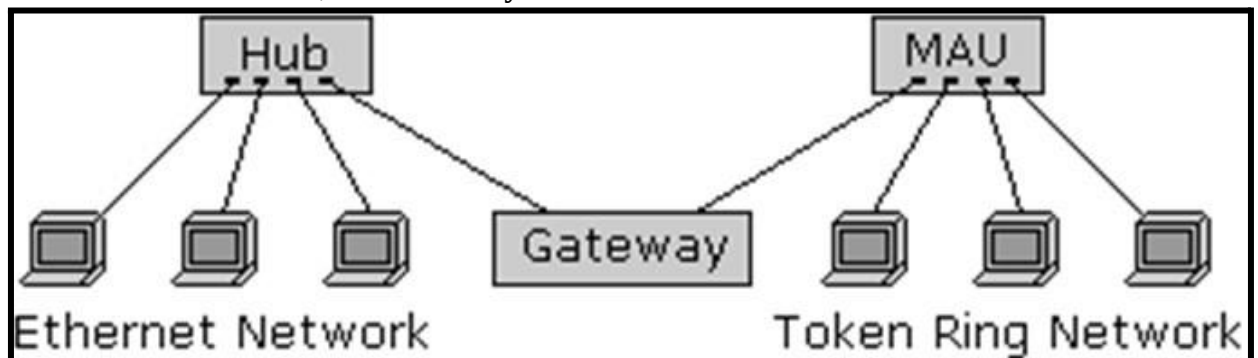


f. Gateway

Gateway is a device that **connects dissimilar networks**.

Establishes intelligent connection between a local network and external networks with completely different structures.

Gateway is the **ISP(Internet Service Provider)** that connects the user to the internet. **Gateway** can also act as **Firewall, API Gateways** etc.



4. Network Topologies

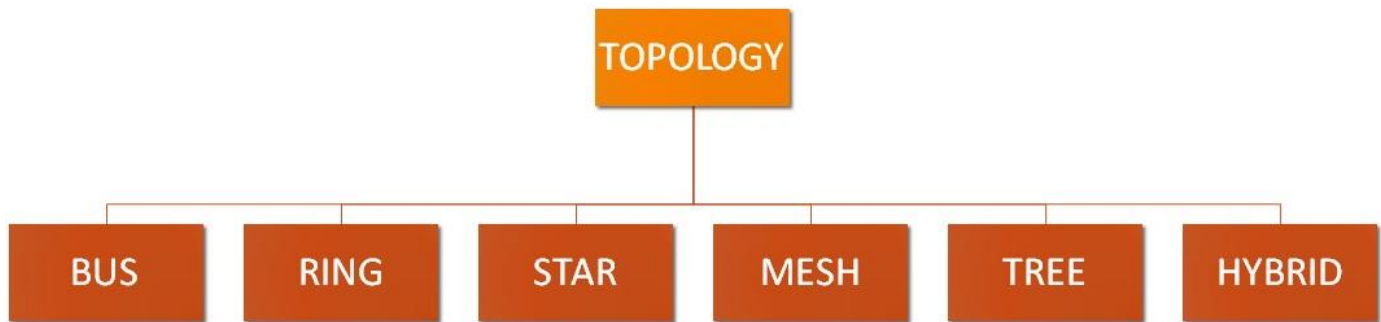
The term **Network Topology** defines the **Geographic, Physical or logical arrangement of computer and networking devices** .

The **pattern of interconnection of nodes** on a network is called the **topology**.

FACTORS for choosing a Topology of a network are:-

- **Cost:** which offers minimum installation cost based on the network under consideration.

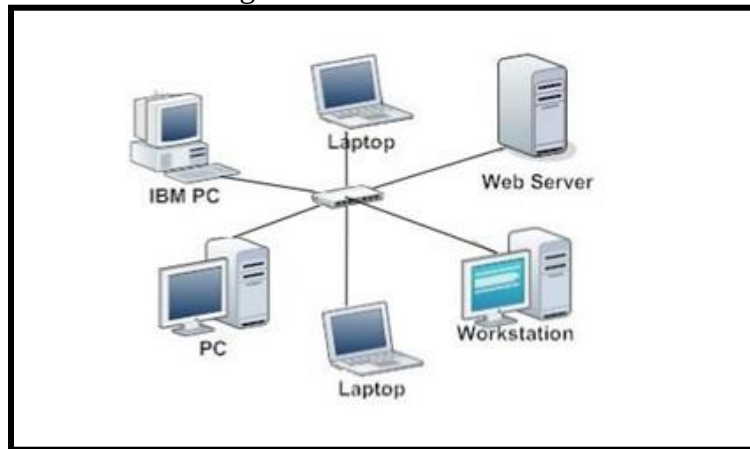
- **Flexibility:** Can offer easy movement of existing nodes and adding new ones.
- **Reliability:** Offers least failure.



a. Star

Each node connected directly to a central network hub or concentrator. Data passes through the hub or concentrator before continuing.

The hub or concentrator manages and controls all functions of the network.



Advantages:

- Easy to install.
- No disruptions to the network when connecting or removing devices.
- Easy to detect faults and to remove parts.

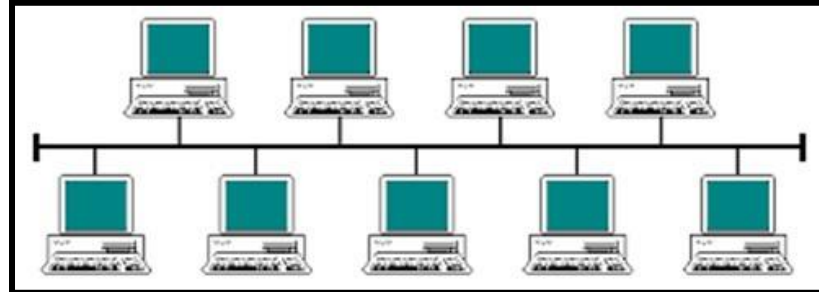
Disadvantages:

- Requires more cable length than a BUS/RING topology.
- If the hub or concentrator fails, nodes attached are disabled.
- More expensive than linear bus topologies because of the cost of the concentrators.

b. Bus

It consists of **one continuous length of cable (trunk)** that is shared by **all the nodes in the network**.

It has a **terminating resistor (terminator)** at **each end** that absorbs the signal when it reaches the end of line.



Advantages:

- Easy to connect a computer.
- Requires less cable length.
- Failure of one node does not affect the network functioning.

Disadvantages:

- Entire network shuts down if there is a break in the main cable.
- Terminators are required at both ends.
- Difficult to identify the problem.
- Nodes must be intelligent.

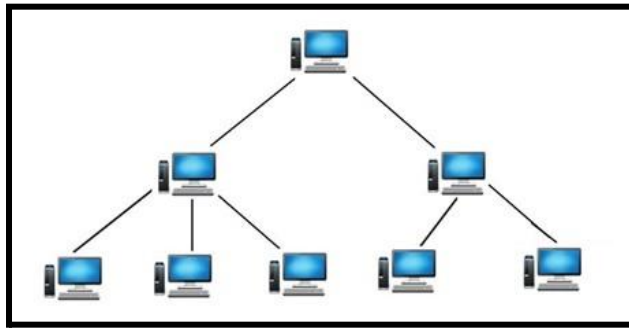
c. Tree

A tree topology combines characteristics of linear BUS and STAR topologies.

- **Inverted tree like structure.**
- It consists of groups of star-configured workstations connected to a linear bus backbone cable.

Advantages:

- Point-to-point wiring for individual segments
- It is highly flexible
- Centralized monitoring

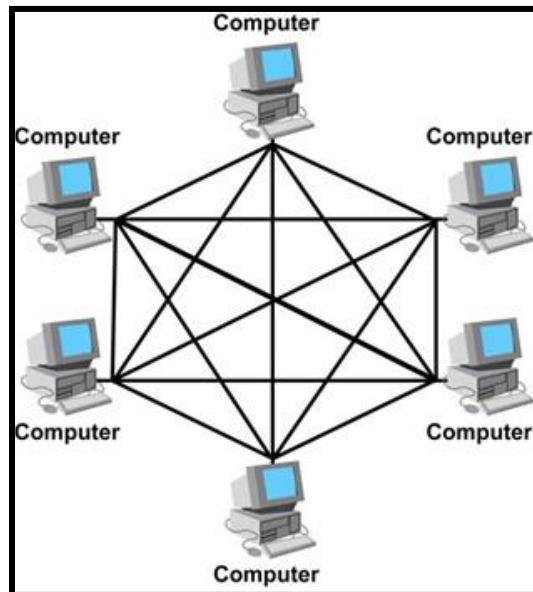


Disadvantages:

- If the backbone line breaks, the entire segment goes down.
- It is difficult to configure the network, if there is a single point of failure.
- More wire is required than other topologies

d. Mesh

Each node is connected to another Node.



Advantages:

- It is robust.
- Provides security and privacy.
- Fault is diagnosed easily.

Disadvantages:

- The cost to implement is higher than other network topologies.
- Installation and configuration is difficult.



1. Introduction to Internet
2. URL
3. WWW and its applications- Web, email, Chat, VoIP.
4. Website: Introduction to website and webpage, static vs dynamic web page, web server and hosting of a website.
5. Web Browsers: Introduction, commonly used browsers, browser settings, add-ons and plug-ins, cookies.
6. Most Important Questions and Answers

1. Introduction to Internet

The **Internet** is composed of hundreds of thousands of interconnected networks. A network is a group of connected, communicating devices such as computers and printers.

2. URL (Uniform Resource Locator)

- URL stands for **Uniform Resource Locator**.
- It was **invented** with the development of WWW by **Tim Berners-Lee**.
- A URL is a web address of a given unique resource on the Web.
- It is the mechanism used by browsers to retrieve any published resource on the web.
- Ex:- <https://www.cbsecsip.in/2020/11/mysql-aggregate-functions.html>



- URLs are used most commonly to reference **web pages (http)**.
- But it is also used for **file transfer (ftp)**, **email (mailto)**, **database access (JDBC)**, and many other applications.
- **Example:**
 - ftp://ftp.cbsecsip.in
 - jdbc:mysql://127.0.0.1:8080/mydb

3. WWW and its applications

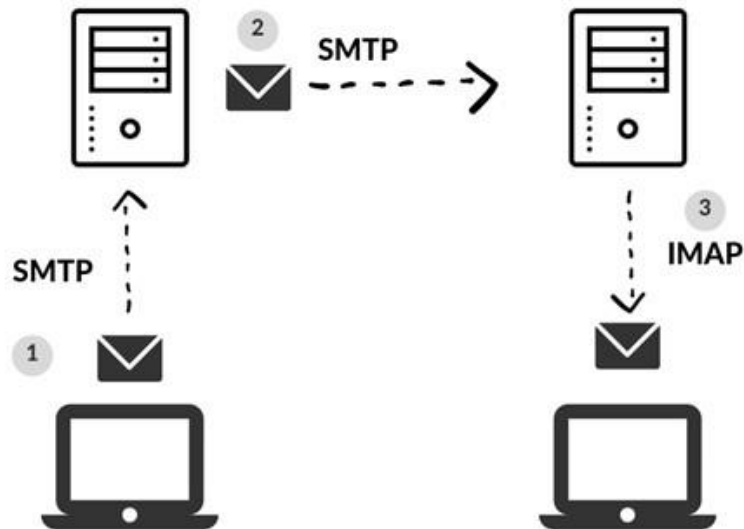
a. WEB

It is an information system where documents and other web resources are identified by **Uniform Resource Locators (URLs)**, such as <https://example.com/>). These resources may be **interlinked by hypertext**, and are accessible over the **Internet**.

- The World Wide Web (WWW), commonly known as the Web.
- WWW is a set of protocols that allows you to access any document on the Net through a naming system based on URLs.
- WWW also specifies a HTTP (Hypertext Transfer Protocol) to request and send documents over the internet.
- **World Wide Web (WWW)** was invented by **Tim Berners-Lee** in the year **1989**.
- **WWW is a small part of the internet, not the internet.**

b. Email

Electronic mail (email or e-mail) is a method of exchanging messages ("mail") between people using electronic devices. Eg. Gmail, rediffmail.



c. Chat

Online Chat May refer to any kind of **communication over the Internet that offers a real-time transmission of text messages** from sender to receiver. Eg. Yahoo Messenger.

d. VoIP

VoIP (Voice over Internet Protocol) is the technology that **converts your voice into a digital signal**, allowing you to make a call directly from a computer, VoIP phone, or other data-driven devices. You may also hear it referred to as **IP telephony, internet telephony, broadband telephony, or broadband phone service**. Eg. Skype

Important Abbreviations

1. **NIU:** Network Interface Unit
2. **MAC:** Media Access Control
3. **TCP/IP:** Transmission Control Protocol/Internet Protocol
4. **PAN:** Personal Area Network
5. **LAN:** Local Area Network
6. **MAN:** Metropolitan Area Network
7. **WAN:** Wide Area Network
8. **Mbps:** Megabits per sec
9. **EMI:** Electromagnetic Interference
10. **RJ:** Registered Jack
11. **Wi-Fi:** Wireless Fidelity
12. **VPN:** Virtual Private Network
13. **NIC:** Network Interface Card
14. **DNS:** Domain Name System
15. **ISP:** Internet Service Provider
16. **URL:** Uniform Resource Locator
17. **HTTP:** HyperText Transfer Protocol
18. **FTP:** File Transfer Protocol

4. Website

a. Introduction

Website:

A website is a **collection of web pages and related content** that is identified by a common domain name and published on at least one web server.

Ex:- cbsecsip.in, wikipedia.org, google.com, and amazon.com.

All publicly accessible websites collectively constitute the World Wide Web.

Web Page:

A **web page (or webpage)** is a specific collection of information provided by a website and displayed to a user in a web browser.

Web pages mainly text files **written in Hypertext Markup Language (HTML)**. Web Page also uses **Javascript and CSS (Cascading Style Sheets)**.

b. Static vs Dynamic web page

Static Web Page:

- It is delivered to the user's web browser exactly as stored.
- It displays the same information for all users, from all contexts. ● Ex:- **Wikipedia**

Dynamic Web Page:

- It is generated by a web application.
- The content and layout of dynamic page changes during runtime.
- Ex:- **YouTube**

c. Web server

Web server is a computer where the web content is stored. It can satisfy client requests on the World Wide Web.

Web server is used to host the web sites but there exists other web servers also such as gaming, storage, FTP, email etc.

There are **two types of Web Server:**

- **Static Web Server:** Provides Static Web Page.
- **Dynamic Web Server:** Provides Dynamic Web page

d. Hosting of a website

Web Hosting is a type of Internet hosting service that allows individuals and organizations to make their website accessible via the World Wide Web.

- Website is hosted on special computers called Web Servers. To Host any website we need Web Server and Web Space.
- Web Hosting provides various services like FTP Upload, Email Account, Website building Tools, Database etc.
- Some Web Hosting Providers are Go Daddy, Hostinger, Blue Host etc.

5. Web Browsers

a. Introduction

- A web browser is a software application for accessing information on the World Wide Web.
- When a user requests a web page from a particular website, the web browser retrieves the necessary content from a web server and then displays the page on the user's device.

b. commonly used browsers

The most used browser is **Google Chrome**, with a 64% global market share on all devices, followed by **Safari** with 18%.other web browser are **Mozilla Firefox ,Microsoft Edge, Internet Explorer, Opera** etc.

c. browser settings

- Depending on the browser, the menu may be named Settings, Options, or Preferences.

- The menu has different types of settings. For example, **users can change their homepage and default search engine.**
- They also can change default **web page colors and fonts.** Various **network connectivity and privacy settings** are also usually available.

d. add-ons and plug-ins

- A plug-in, add-on, or extension are generally the same thing.
- Web browser plugins and add-ons are **programs integrated into your web browser for extending its capabilities and adding some features.**
- A browser plug-in expands a web browser's functionality by installing specific toolbars, adding buttons, links, or other useful functions, like pop-up blocking, ad blocking.
- It allows third-party developers to create specific functions

Problems with Extension

- Access to sensitive data, such as browsing history
- Ability to alter some browser settings
- Some may be malicious, they can install adware, install spyware, or redirect your browser to a specific site.
- Some are often installed without explicit user consent and frequently have no easily-accessible way to uninstall them.

e. Cookies

An **HTTP cookie** (also called **web cookie, Internet cookie, browser cookie, or simply cookie**) is a **small piece of data stored on the user's computer by the web browser while browsing a website.**

- During the course of browsing, cookies received from various websites are stored by the browser.
- Some of them contain login credentials or site preferences. However, others are used for tracking user behavior over long periods of time, so browsers typically provide settings for removing cookies.



1. Digital footprint
2. net and communication etiquettes
3. intellectual property rights (IPR)
4. Plagiarism
5. licensing and copyright
6. free and open source software (FOSS),
7. Cyber Crime - hacking, phishing, cyber bullying,
8. Cyber Laws - overview of Indian IT Act
9. E-waste: hazards and management
10. Awareness about health concerns related to the usage of technology
11. Most Important Questions and Answers

1. Digital Footprint -

What is the Digital footprint ?

- A **digital footprint** is the impression you create on the internet through your online activity, which includes **browsing, interactions with others, and publication of content**.
- In other words, it is the trail of data – intentional and unintentional - you leave behind while surfing the internet.
- **Digital footprint** or **digital shadow** refers to the trail of data left behind through the use of the Internet or on digital devices.
- The use of a digital footprint has both **positive** and **negative consequences**.

How Digital Footprints are Created?

- Visiting Websites And Online Shopping
- Online Searching
- Posting on Social Media, blogs etc.
- Online Image and Video Upload
- Communicating Online (Ex:- Chat, Email etc.) → Any activity you perform Online etc.

How digital footprints can affect you?

- **Privacy Issues:** Digital footprints are a privacy concern because they are a set of traceable actions, contributions, and ideas shared by users. It can be tracked and can allow internet users to learn about human actions.
- **Cyber Vetting:** where interviewers could research applicants based on their online activities.
- **Target advertisement:** used by marketers in order to find what products a user is interested in, or to inspire one's interest in a certain product based on similar interests.
- **Less/More Opportunities** depends upon your Positive/Negative Digital Footprints.

2. Net and communication etiquettes

Online etiquette or '**netiquette**' refers to the **dos and don'ts of online communication**.

Online Communication can be done through various means available online, most popular online mediums are Snapchat, Instagram, WhatsApp etc.

You Should Follow:

- Be respectful.
- Be aware of how your comments might be read
- Be careful with humor and sarcasm
- Think about who can see what you have shared
- Remember to check friend requests and group invites before accepting them
- Take time to have a read of the rules of conduct/ community standards ● Be forgiving do not take fight online

3. Intellectual Property Rights

- **Intellectual property rights (IPR)** are the **rights given to persons over the creations of their minds**.

- It usually gives the creator an exclusive right over the use of his/her creation for a certain period of time.
- Intellectual property rights can be claimed at national/regional Level.
- For Example, Coca-Cola recipe, Nike Logo, Any software, Website content etc.

4. Plagiarism

Plagiarism is stealing someone else's intellectual work (can be an idea, literary work or academic work etc.) and representing it as your own work without giving credit to the creator or without citing the source of information.

Plagiarism is an offence because **it involves copyright infringement and not giving the credit to the author.**

5. Licensing and Copyright

Copyright (or author's right) is a legal term used to describe the rights that creators have over their literary and artistic works.

Copyright protections are **automatic**; once you create something, it is yours.

Works covered by copyright range from **books, music, paintings, sculpture, and films, to computer programs, databases, advertisements, maps, and technical drawings.**

Typically, the public law duration of a **copyright expires 50 to 100 years after the creator dies**, depending on the jurisdiction.

Example:- The video of cbseclip on YouTube are Copyrighted under CBSE CS & IP.

Software Licencing

A software license is a contract between the entity that created and supplied an application, underlying source code, or related product and its end user. The license is a text document designed to protect the intellectual property of the software developer and to limit any claims against them that may arise from its use.

Most software falls under one of two categories that have distinct differences in how they are viewed under copyright law:

Proprietary – also referred to as “closed source”

Free and open-source software (FOSS) – referred to as “open source”

6. Free and open source software (FOSS)

Free and open-source software (FOSS) is software that can be classified as both **free software** and **open-source software**.

That is, **anyone is freely licensed to use, copy, study, and change the software in any way, and the source code is openly shared so that people are encouraged to voluntarily improve the design of the software.**

FOSS is also called FLOSS (Free Libre and Open Source Software or Free Livre and Open Source Software). Libre (Spanish Word) , Livre(Portuguese Word).

Free software:- “Free Software” is a matter of liberty not price. It provides four freedom to Run, Study how the program works, Redistribute copies, Release improvement to Public.

Open-Source software:- In this type of software the source code is freely available.

7. Cyber Crime - hacking, phishing, cyber bullying

Cyber Crime

Cyber Crime, or computer-oriented crime, is a crime that **involves a computer and a network**.

The computer may have been used in the commission of a crime, or it may be the target. Cybercrime may threaten a person, company or a nation's security and financial health.

Phishing

Phishing refers to a criminally fraudulent process of **attempting to acquire sensitive information** such as usernames, passwords, credit card information, account data etc.

In phishing , **an imposter uses an authentic looking email or website to trick recipients** into giving out sensitive personal information.

Hacking

Hacking is an **unauthorized entry into a network or a computer to steal or manipulate information, data or files**.

The person involved in this process is named as a hacker.

Computer hacking is done using several types of programs such as Rootkit, Trojan, Keylogger etc.

How to Prevent Hacking?

- Download software from authorized websites
- Do not click on random email attachments
- Scan all types of hard drives before running
- Abstain from keeping easy passwords
- Never store or share your login information

Cyber Bullying

Harassing, demeaning, embarrassing, defaming or intimidating someone using modern technologies like Internet, cell phones, instant messengers, social networks etc. is called Cyber Bullying

8. Cyber Laws - overview of Indian IT Act

Cyber law, also known as cyber crime law, is **legislation focused on the acceptable behavioral use of technology including computer hardware and software, the internet and networks**.

- Cyber law helps protect users from harm by enabling the investigation and prosecution of online criminal activity.
- It applies to the actions of individuals, groups, the public, government and private organizations.
- The **Information Technology Act, 2000** (also known as ITA-2000, or the IT Act) is an Act of the Indian Parliament notified on 17 October 2000. It is the primary law in India dealing with cybercrime and electronic commerce.
- A major amendment was made in 2008 and now it is called **IT (Amendment) Act 2008**.

9. E-waste: hazards and management

E-waste or Waste Electrical and Electronic Equipment (WEEE) are loosely discarded, surplus, obsolete, broken, electrical or electronic devices.

- E-waste poses many problems related to the **environment** and **health**.
- Electronic equipment contains many hazardous metallic contaminants such as *lead, cadmium, mercury* and *beryllium* and *brominated flame-retardants*.

Environmental Hazards:

When e-waste is warmed up, toxic chemicals are released into the air damaging the atmosphere. When electronic waste is thrown away in landfills their toxic materials seep into groundwater, affecting both land and sea animals.

Health Hazards:

Different metals compositions causes many Health Hazards:

- **Lead:** Damage to central and peripheral nervous systems, blood systems, and kidney damage. Adverse effects on brain development of children; causes damage to the circulatory system and kidney.
- **Cadmium:** Toxic irreversible effects on human health. Accumulates in kidney and liver. Causes neural damage.
- **Mercury:** Chronic damage to the brain. Respiratory and skin disorders due to bioaccumulation in fishes.
- **Beryllium:** Carcinogenic (lung cancer). Inhalation of fumes and dust causes chronic beryllium disease or berylliosis.
- **Brominated flame-retardants:** Disrupt endocrine system functions

E-Waste Management

1. Dismantling
2. Segregation of recyclable and non-recyclable materials
3. Refurbishment and reuse
4. Recycle/recovery of valuable materials
5. Treatment/disposal of dangerous materials and waste

10. Awareness about health concerns related to the usage of technology

Excessive use of technology can have Physical and Physiological effects on humans.

Physical health effects:

- Eyestrain
- Poor posture
- Sleep problems
- Reduced physical activity

Psychological effects:

- Isolation
- Depression and anxiety